



基於深度學習之 氣胸X光影像分割

SIIM-ACR Pneumothorax Segmentation Challenge

組員：B1228005胡樂麒、B1228010李怡萱、B1228011劉姮廷、
B1228021邱庭俞、B1228039蔡勇濱



摘要



- **背景：** 氣胸（Pneumothorax）是危急胸腔疾病，早期診斷困難。
- **目的：** 利用 SIIM-ACR 資料集進行語義分割（Semantic Segmentation）。
- **方法：** 本實作採用 Unet++ 架構，搭配不同編碼器（Encoder），並引入醫學影像專用的強度預處理流程（Intensity Pipeline）。
- **結果：** 透過混合損失函數（Mixed Loss）與閾值優化（Threshold Tuning），模型能有效標記出氣胸區域。



緒論



- **研究背景：** 微小氣胸特徵不明顯，易錯過（氣胸區域對比度低、易與肋骨混淆）。
- **任務定義：** 這是一個「二元分割任務」（Binary Segmentation），目標是輸出與原圖大小相同的 Mask（遮罩），標示出氣胸位置。
- **資料集介紹：**
 - **來源：** siim-acr-pneumothorax-segmentation-data
- **數據統計：**
 - **規模：** > 12,000 張 DICOM 格式胸部 X 光影像
 - **標註：** 遊程編碼 (RLE) 格式的分割遮罩



資料探索與前處理

怎麼處理資料？

資料分析



- 資料集：

- test/train比例(1,377 VS. 10,712)
- 有氣胸/無氣胸比例(3,286 VS. 8,296)

```
train_rle['EncodedPixels'] = train_rle['EncodedPixels'].str.strip()
print("沒有氣胸的數量：", (train_rle['EncodedPixels'] == '-1').sum())
print("有氣胸的數量：", (train_rle['EncodedPixels'] != '-1').sum())
```

沒有氣胸的數量： 8296
有氣胸的數量： 3286

```
import glob
train = sorted(glob.glob("../input/siim-acr-pneumothorax-segmentation-data/pneumothorax/dicom-images-tra
test = sorted(glob.glob("../input/siim-acr-pneumothorax-segmentation-data/pneumothorax/dicom-images-test
print("DICOM 的訓練資料：", len(train))
print("DICOM 的測試資料：", len(test))
```

DICOM 的訓練資料： 10712
DICOM 的測試資料： 1377



資料分析



- dicom資料集：

- 資料包含:影像類型、檢查部位、病人與醫療背景、影像檔案技術參數等

```
data = pydicom.dcmread(test[5])  
print(data)
```

```
Dataset.file_meta -----  
(0002, 0000) File Meta Information Group Length  UL: 200  
(0002, 0001) File Meta Information Version       OB: b'\x00\x01'  
(0002, 0002) Media Storage SOP Class UID        UI: Secondary Capture Image Storage  
(0002, 0003) Media Storage SOP Instance UID     UI: 1.2.276.0.7230010.3.1.4.8323329.5801.1517875190.850480  
(0002, 0010) Transfer Syntax UID                UI: JPEG Baseline (Process 1)  
(0002, 0012) Implementation Class UID           UI: 1.2.276.0.7230010.3.0.3.6.0  
(0002, 0013) Implementation Version Name        SH: 'OFFIS_DCMTK_360'  
-----
```



DICOM 圖片



Pneumothorax



No Pneumothorax



DICOM 影像處理

- 說明：我們參考老師的建議，不轉換使用 JPG，而是處理 DICOM 原始數據。
- 技術細節：讀取 DICOM 的 RescaleSlope (斜率) 與 RescaleIntercept (截距) 進行線性轉換，還原真實像素值。
- 處理 PhotometricInterpretation 為 MONOCHROME1 的影像 (進行反相處理)，確保所有影像的黑白定義一致 (空氣為黑，骨骼為白)。

```
# rescale (若存在)
slope = float(getattr(dcm, "RescaleSlope", 1.0))
intercept = float(getattr(dcm, "RescaleIntercept", 0.0))
img = img * slope + intercept

# MONOCHROME1 需要反相 (常見於醫學影像)
if getattr(dcm, "PhotometricInterpretation", "") == "MONOCHROME1":
    img = img.max() - img

return img
```

視窗化與標準化

- 說明：為了去除極端亮點或噪點。
- 技術細節：我們使用了 Percentile Clipping (1% - 99%)，將像素值限制在第 1 到第 99 百分位數之間，再將影像數值標準化至[0, 1]的範圍。

```
def medical_intensity_pipeline(img, clip=(1, 99), eps=1e-6):  
    lo, hi = np.percentile(img, clip)  
    img = np.clip(img, lo, hi)  
    img = (img - img.min()) / (img.max() - img.min() + eps)  
    return img.astype(np.float32)
```

資料集切分

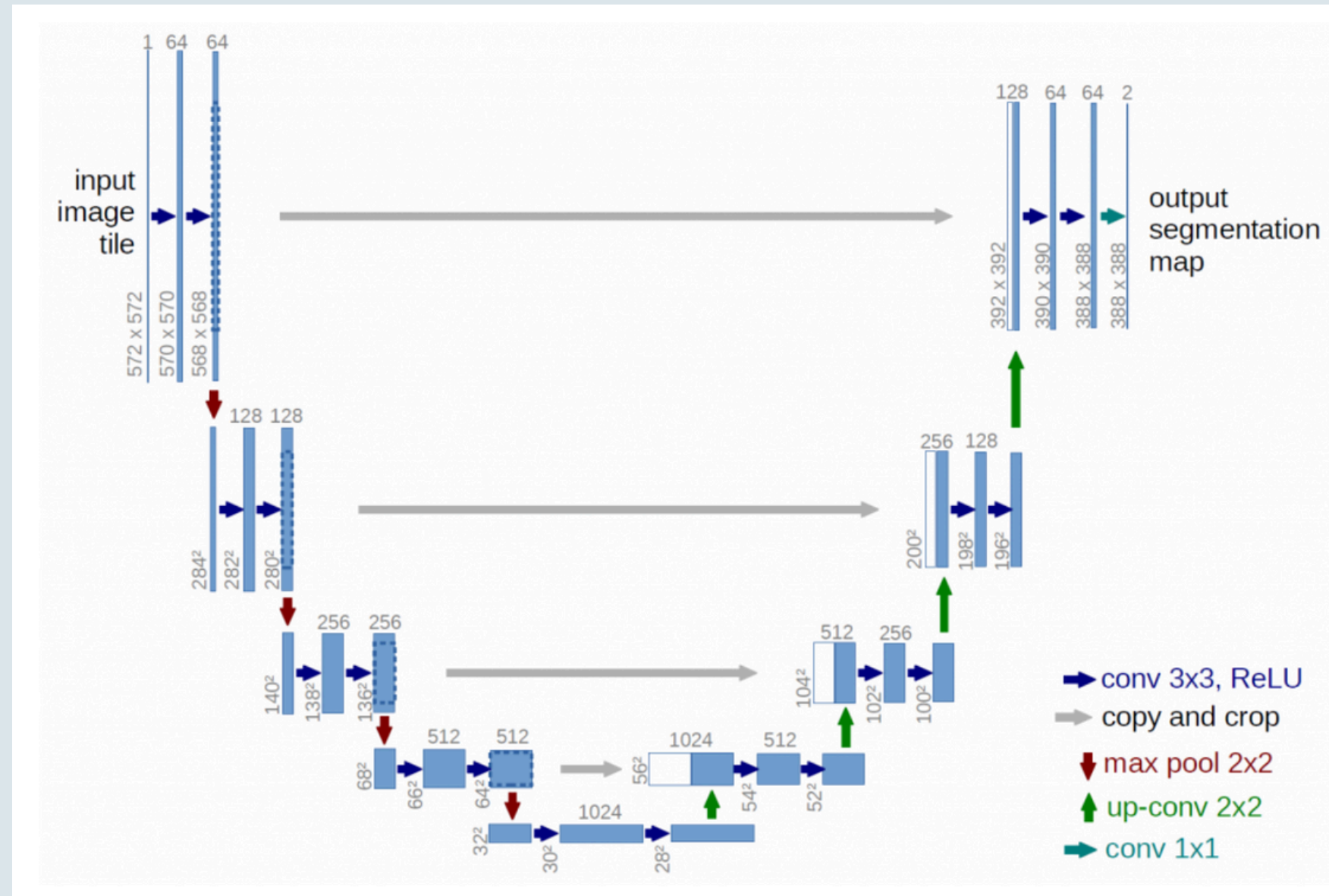
- 說明：我們使用 Stratified K-Fold (分層交叉驗證)，K=5。
 - 這確保了訓練集和驗證集中「有氣胸」的樣本比例是固定的，解決資料分佈不均問題。
 - 我們知道資料集有分佈不均問題，但因為我們嘗試做數據增強時遇到問題，因此沒有使用數據增強辦法。

```
# === 修改核心：使用 Stratified K-Fold ===  
# 設定 K=5，並固定 random_state 以確保可重現  
N_SPLITS = 5  
FOLD_TO_USE = 0 # 我們使用第 0 個 fold 作為驗證集  
  
skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=CONFIG['seed'])  
  
# 根據 'has_pneumo' 欄位進行分層  
# skf.split 回傳的是 index  
# 我們只需要取得第 FOLD_TO_USE 組的 train/val index  
for fold, (train_idx, val_idx) in enumerate(skf.split(full_df, full_df['has_pneumo'])):  
    if fold == FOLD_TO_USE:  
        train_df = full_df.iloc[train_idx].reset_index(drop=True)  
        val_df = full_df.iloc[val_idx].reset_index(drop=True)  
        break
```

主架構選擇

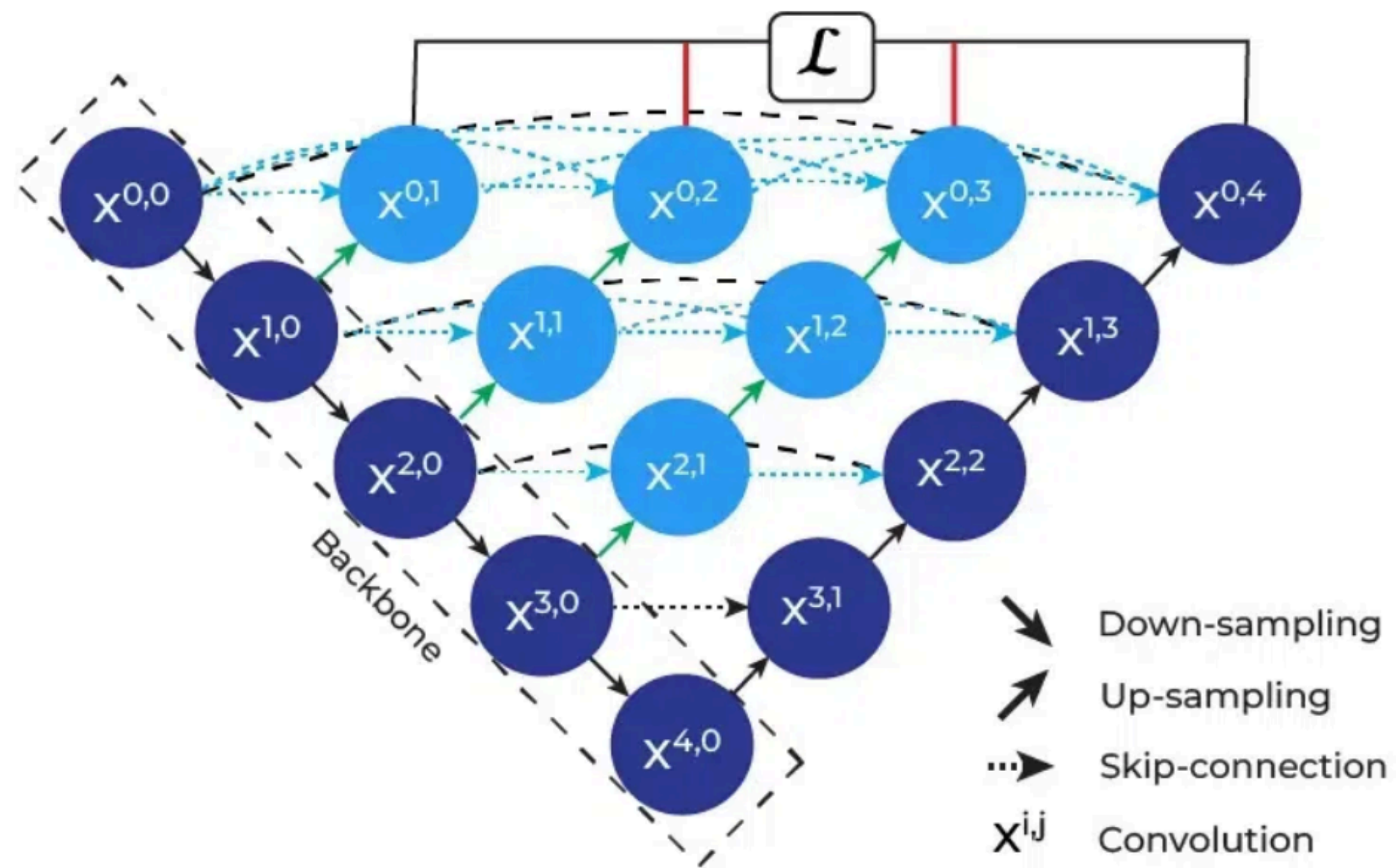
U-Net、U-Net++選哪個？

U-Net



- 左邊 (Encoder/Downsampling)：負責提取特徵（例如：這是邊緣、這是紋理），圖片會越來越小。
- 右邊 (Decoder/Upsampling)：負責還原圖片尺寸，並畫出 Mask。
- 灰色箭頭 (Skip Connections)：它把左邊的特徵圖直接複製並拼接 to 右邊對應的層。這讓右邊在還原時，不會忘記原本圖片的細節位置。

U-Net++



U-Net++在原本空空的中間區域，加入了很多卷積層 (Convolution Units) 和連接線。

- 網狀結構：它不只是把左邊複製到右邊，而是讓左邊的特徵經過一系列「過渡」的卷積層處理後，才傳給右邊。
- 特徵融合：原本 U-Net 是「長距離跳躍」，U-Net++ 則是「短距離、多路徑的密集跳躍」。

為什麼選擇 U-Net++?

理由一：縮小「語義落差」

- U-Net 的問題：左邊的特徵（淺層特徵，解析度高但語義簡單）和右邊的特徵（深層特徵，經過高度抽象化）差異很大，解碼器需要花很大力氣去適應。
- U-Net++ 的解法：U-Net++在中間多加了很多層卷積層，讓左邊的特徵逐漸演變，等到傳到右邊時，兩者的特徵已經很接近了，融合起來更自然，模型更容易學習。

為什麼選擇 U-Net++?

理由二：適應不同大小的病灶

- 氣胸 (Pneumothorax) 的範圍變化很大，有時候是整個大面積肺部塌陷，有時候只有小面積肺尖一點點氣體。U-Net++能夠同時捕捉到淺層的細節（適合小氣胸）。

理由三：靈活性

- U-Net++可以在網路的不同深度輸出預測結果。在訓練困難的醫學影像任務中，這有助於改善梯度傳遞，使模型的中間層也能獲得有效訓練。

總結比較

特性	U-Net	U-Net++
結構複雜度	低 (Simple)	高 (Complex, 參數稍多)
跳躍連接	單純的複製貼上 (Copy & Crop)	經過卷積處理的密集連接 (Nested & Dense)
特徵融合難度	較難 (語義落差大)	較容易 (漸進式融合)
醫學影像表現	優秀	通常更優
訓練速度	較快	稍慢 (因為計算量較大)

網路架構與方法

所以我們怎麼做？

模型架構

- 主架構：Unet++ (Nested U-Net)。
- 編碼器 (Backbone)：嘗試多種不同編碼器 (使用 ImageNet 預訓練權重)。

```
self.model = smp.UnetPlusPlus(  
    encoder_name=backbone,  
    encoder_weights="imagenet",  
    in_channels=3,  
    classes=1,  
)
```

損失函數

- 說明：採用混合損失函數 (Compound Loss)。
- 公式： $\text{Loss} = 0.8 * \text{DiceLoss} + 0.2 * \text{BCEWithLogitsLoss}$
- 理由：BCE 負責像素級別的分類準確度，Dice Loss 負責優化預測區域的重疊度 (IoU)，兩者結合能加速收斂。

```
self.dice = smp.losses.DiceLoss(mode="binary", from_logits=True)
self.bce = nn.BCEWithLogitsLoss()
self.dice_w = dice_w
self.bce_w = bce_w

def _loss(self, logits, mask):
    # mask: float32 0/1
    loss_dice = self.dice(logits, mask)
    loss_bce = self.bce(logits, mask)
    return self.dice_w * loss_dice + self.bce_w * loss_bce
```

優化策略

- 優化器：AdamW (解決 Adam 的權重衰減問題，提升泛化能力)。
- 學習率排程：CosineAnnealingLR (餘弦退火)，讓學習率隨訓練週期下降，幫助模型收斂到全域最佳解。
- 混合精度訓練 (AMP)：使用 FP16 進行訓練，減少顯存佔用並加快訓練速度。

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=self.trainer.max_epochs, # 根據總 epoch 數調整
    eta_min=1e-6 # 最小學習率
```

```
def configure_optimizers(self):
    optimizer = torch.optim.AdamW(self.parameters(), lr=self.lr, weight_decay=1e-5)
```

Model Training & Selection Pipeline

我們怎麼設定？

參數設定

- Image size: 512×512
- Batch size: 8
- Learning rate: 1e-4
- num_workers: 4
- Loss: Dice (0.8) + BCE (0.2)

```
CONFIG = {  
    "seed": 42,  
    "debug": False,  
    "img_size": 512,  
    "backbone": "resnet101",  
    "batch_size": 8,  
    "lr": 1e-4,  
    "epochs": 20,  
    "num_workers": 4,  
    "device": "cuda",  
    "dice_w": 0.8,  
    "bce_w": 0.2,  
}
```

訓練策略設計

- Loss function : Dice Loss + BCE Loss

- Dice Loss 可處理前景比例極低的類別不平衡問題
- BCE Loss 提供穩定的梯度，提升訓練穩定性

- Gradient accumulation

- 在顯存限制下模擬較大的 batch size，使訓練更穩定

- Validation Dice 作為監控指標 (Early stopping & Checkpoint)

- 直接對齊 segmentation 任務的評估目標，避免模型過擬合

- Optimization

- AdamW (weight_decay=1e-5)

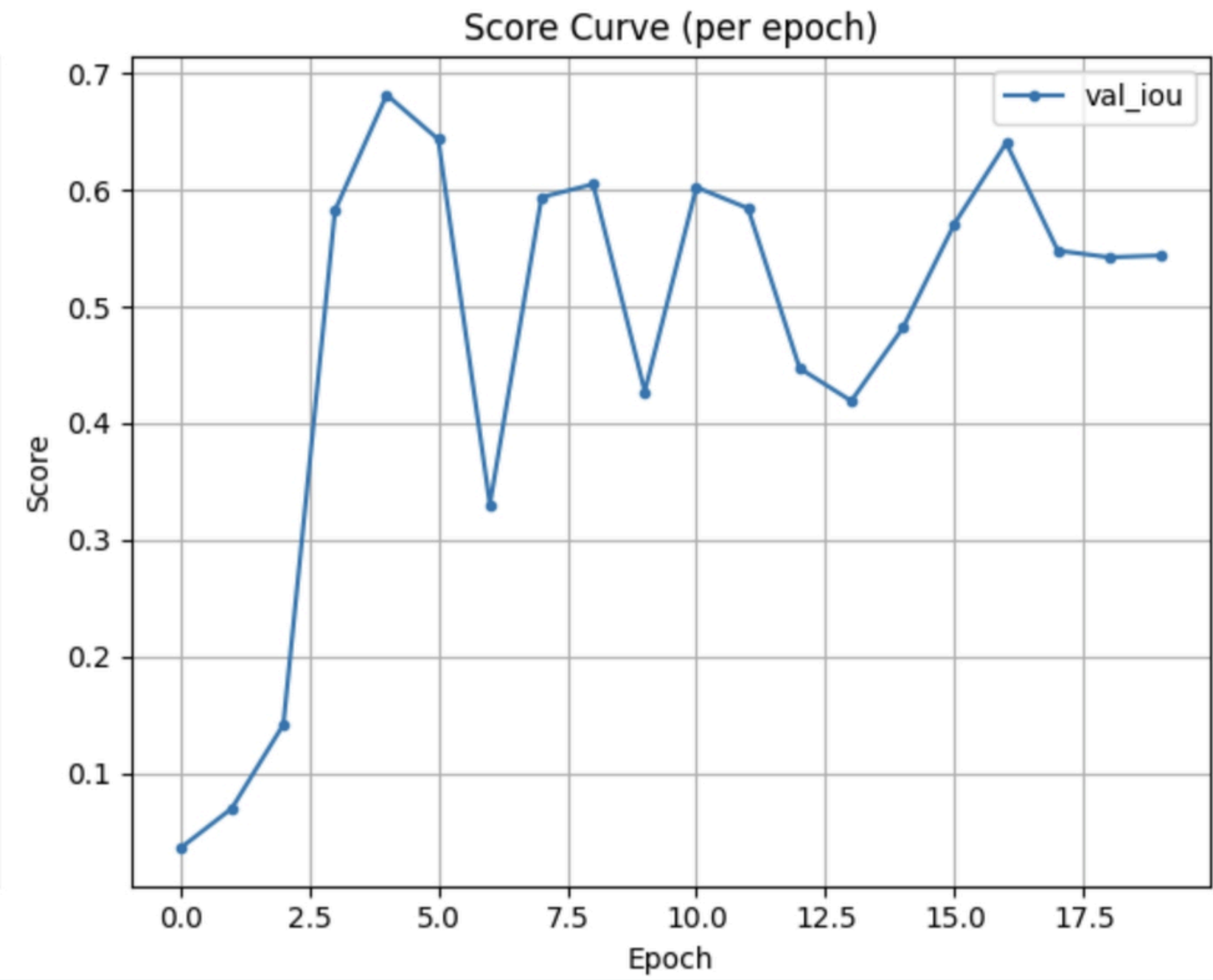
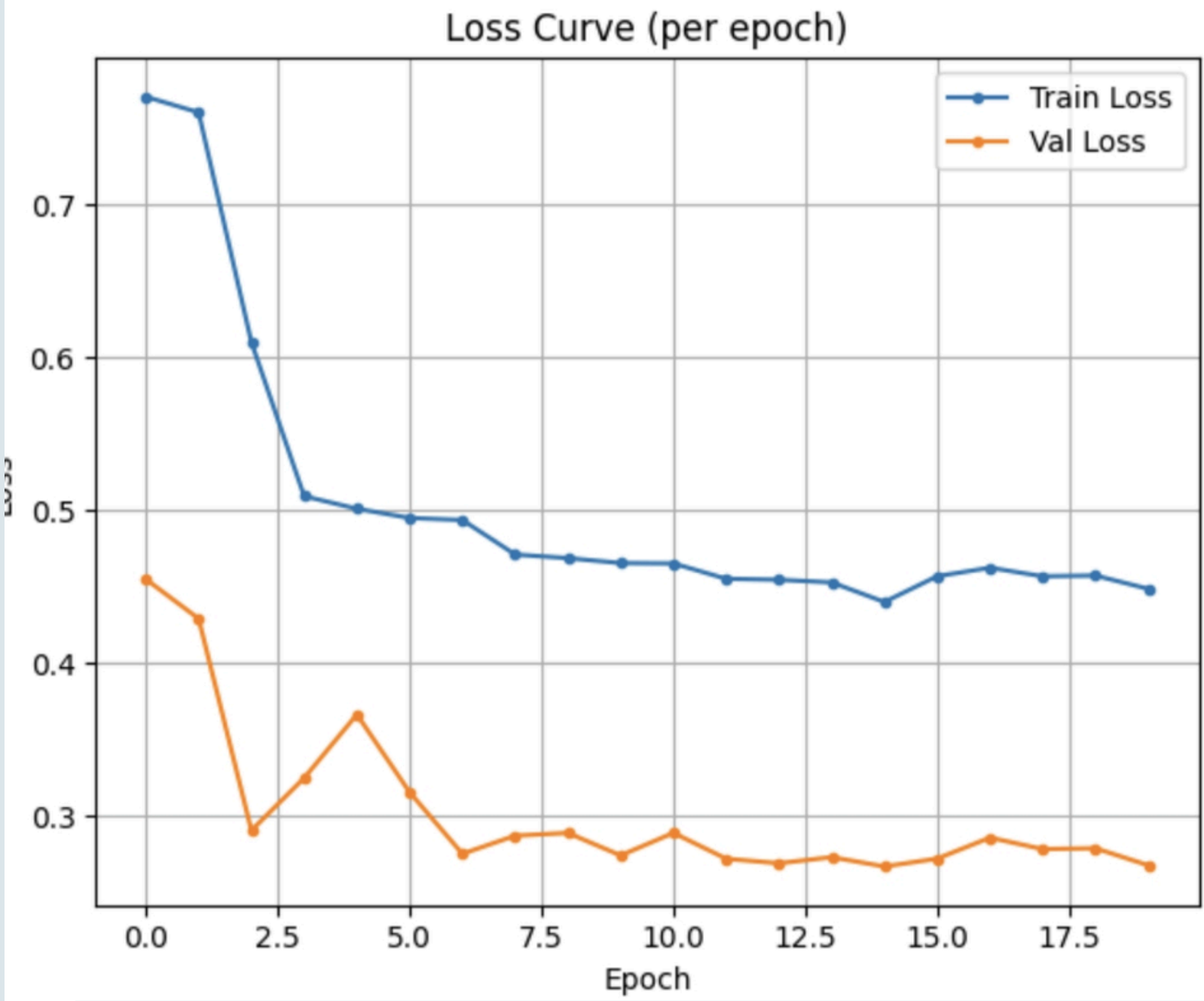
- CosineAnnealingLR (T_max = max_epochs, eta_min=1e-6)

```
trainer = pl.Trainer(  
    accelerator="gpu",  
    devices=1,  
    max_epochs=CONFIG["epochs"],  
    precision="16-mixed",  
    logger=csv_logger,  
    callbacks=[early_stop, ckpt],  
    accumulate_grad_batches=2,  
    gradient_clip_val=1.0,  
    log_every_n_steps=20,  
)
```

實驗結果

不同Backbone有甚麼成果？

backbone : Resnet-50



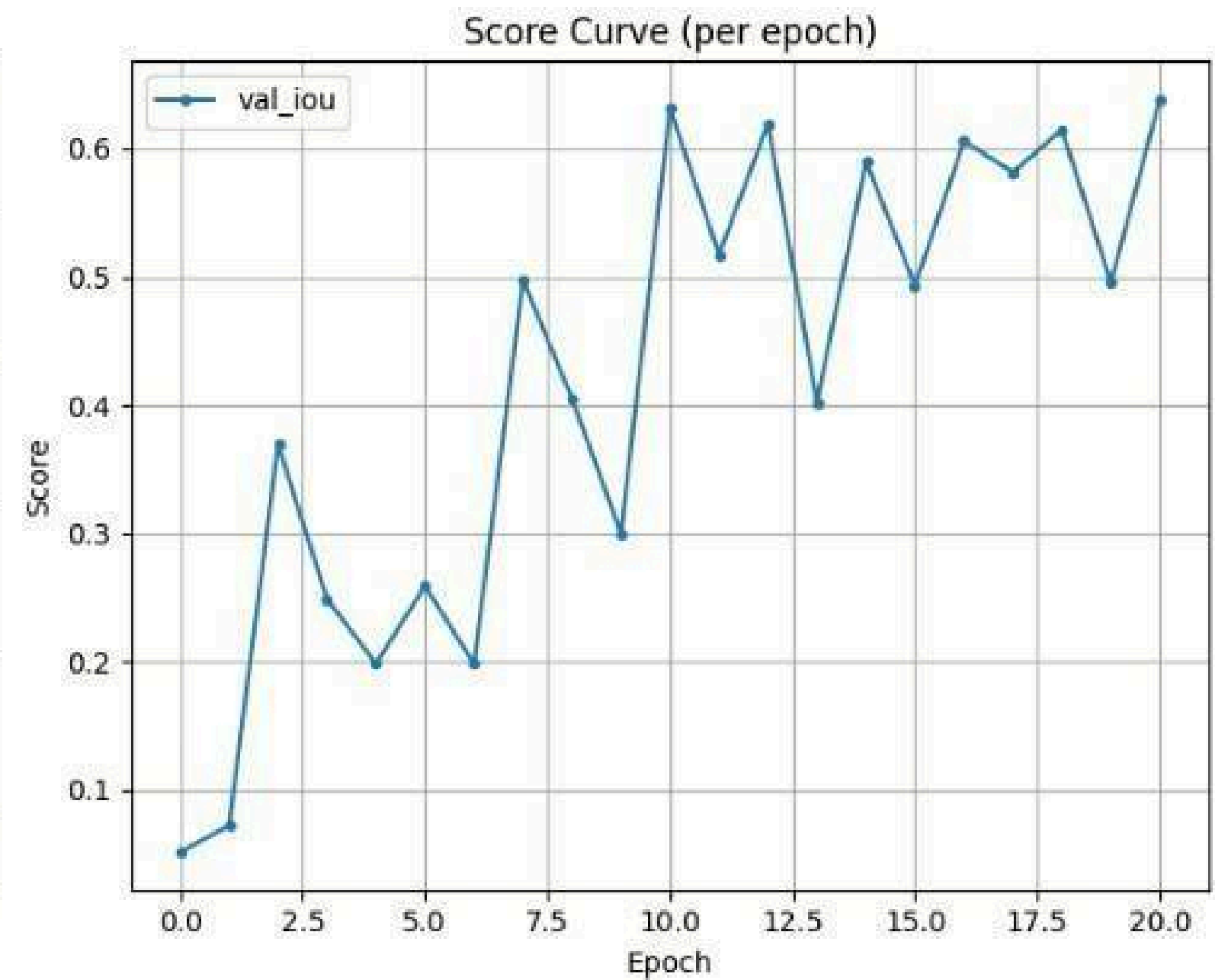
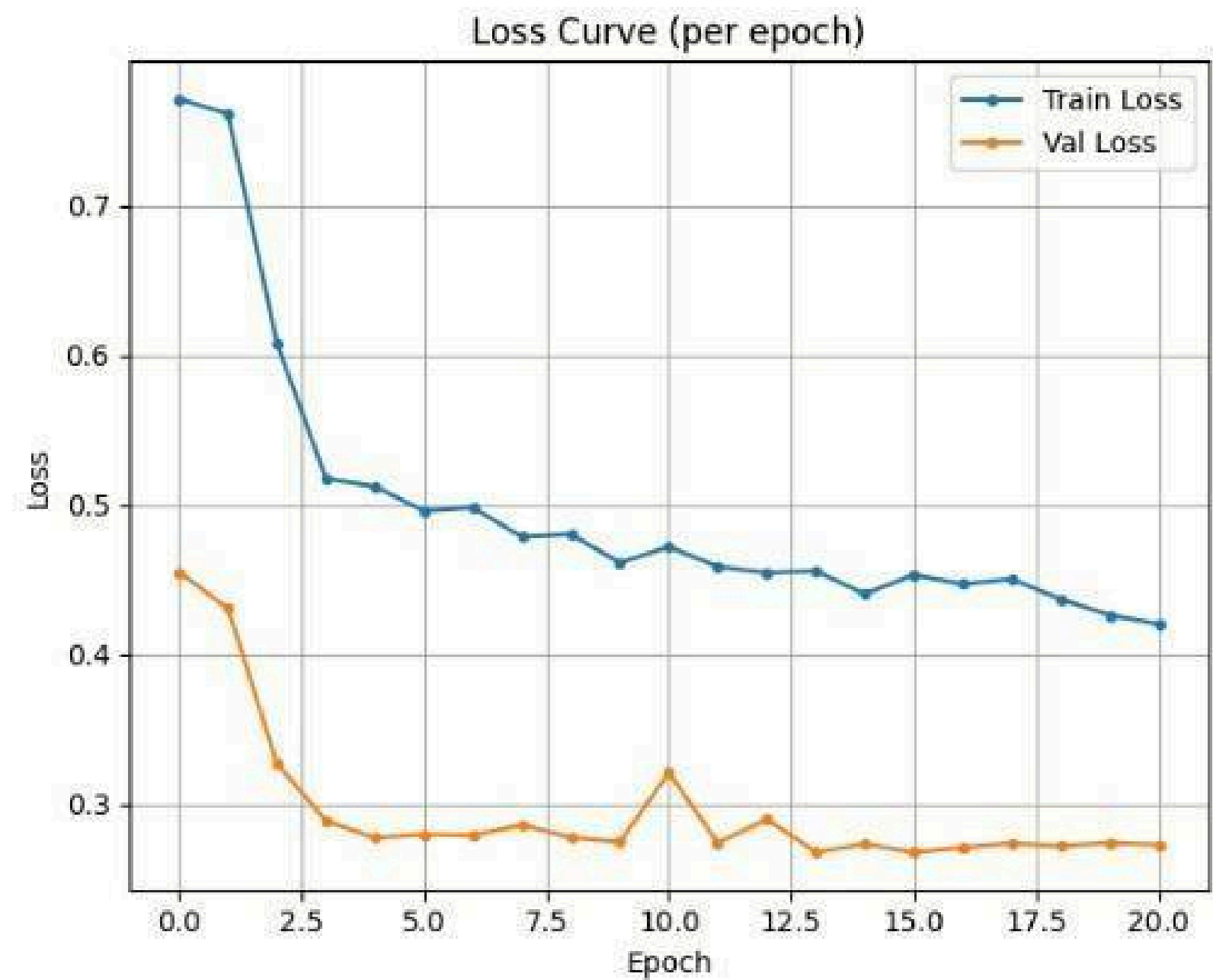
siimacr - Version 3

Complete (after deadline) · 3d ago

0.4606

0.6000

backbone : Resnet-101



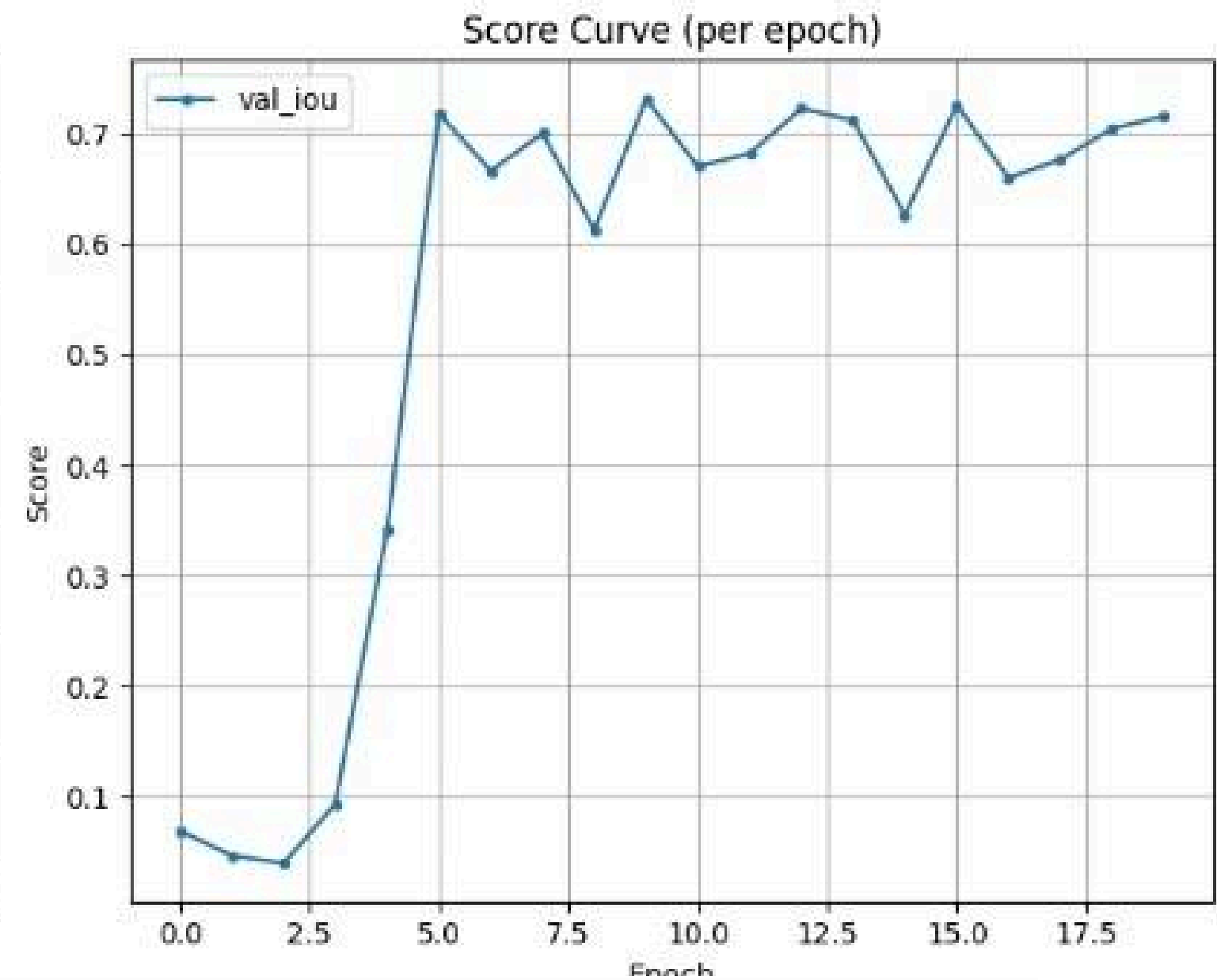
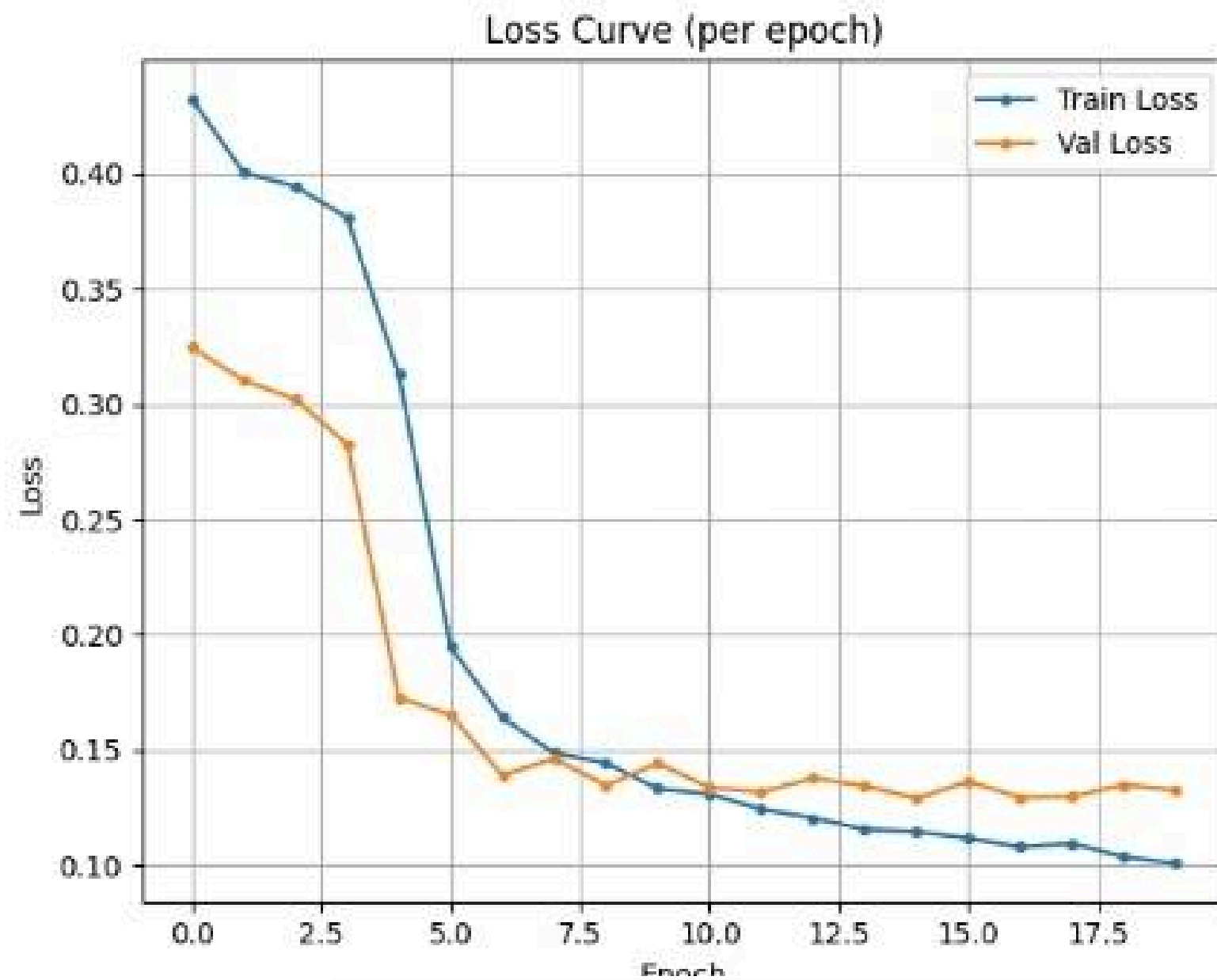
siimacr - Version 4

Complete (after deadline) · 13h ago

0.5028

0.6000

backbone : Efficientnet-b5



tingver - Version 2

Complete (after deadline) · now

0.7740

0.6000

最總選擇

ResNet-50 (Baseline)

- 訓練與驗證 loss 均能穩定下降
- Validation IoU 波動較大，且較早達到效能瓶頸
- 對氣胸這類「區域小、邊界模糊」的病灶表現有限

→ 作為穩定基準模型，但效能上限較低

ResNet-101 (較深模型)

- Validation IoU 峰值略高於 ResNet-50
 - 隨模型深度增加，epoch 間波動明顯
 - 未帶來顯著的穩定性或泛化提升
- 單純增加模型深度，對本任務的改善有限

EfficientNet-B5 (最終選擇)

- 訓練與驗證 loss 收斂快速且一致
 - Validation IoU 整體最高，且波動幅度最小
 - 在模型容量與泛化能力之間取得最佳平衡
- 整體效能與穩定度最佳，最適合作為最終 backbone

觀察結果

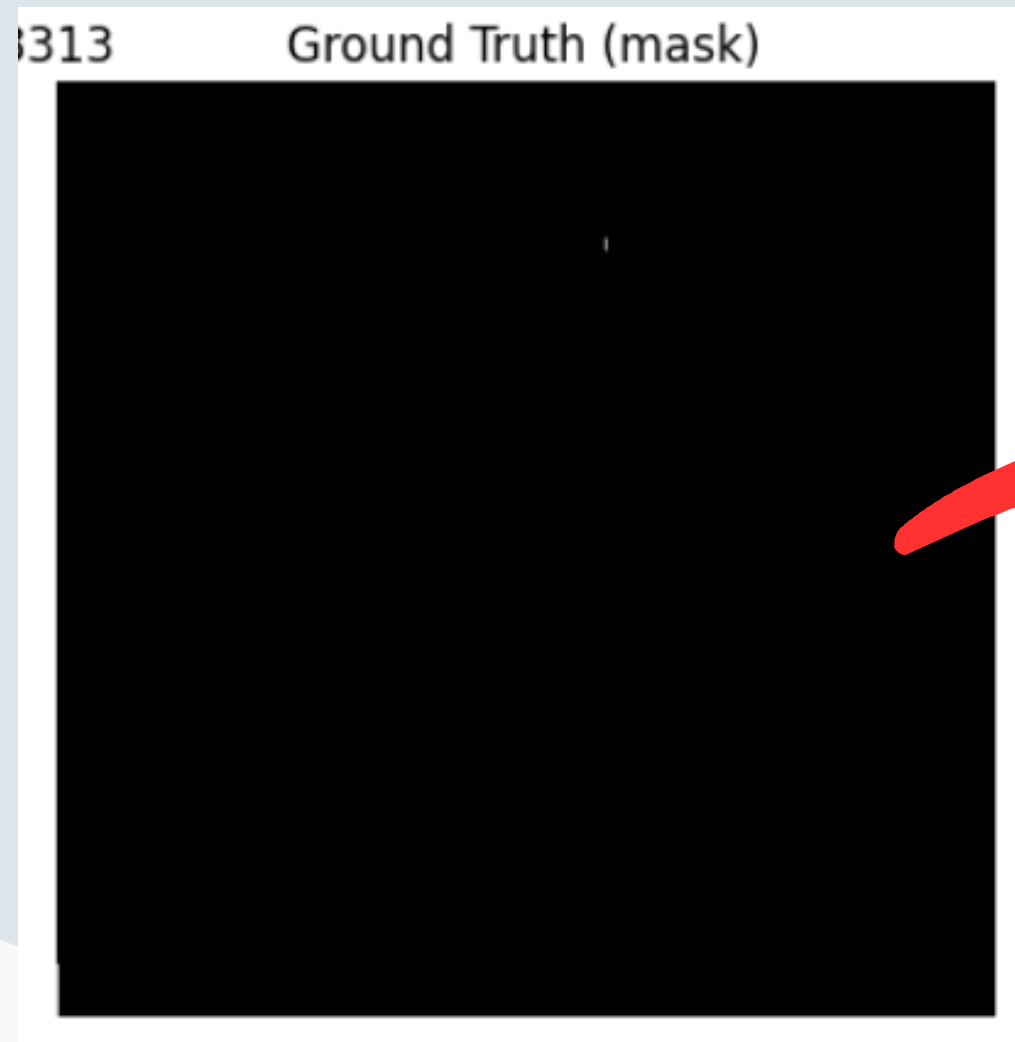
但...我們發現了一些問題

最大問題

- RLE 編碼/解碼邏輯

- ~~「絕對位置」~~ vs. 「相對位置」 -> 使用錯誤!
- 錯誤的邏輯導致模型學習不好，進而最終結果皆為一樣的

0.7740(判斷全黑)

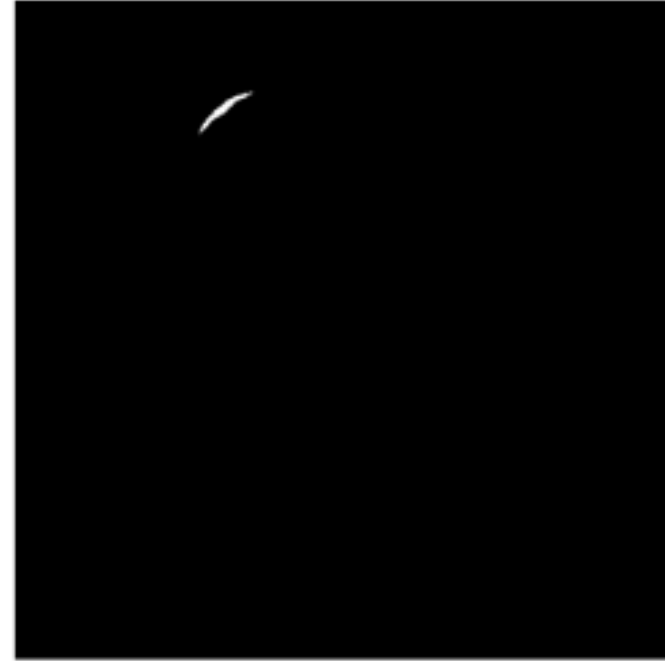


最大問題:修正後

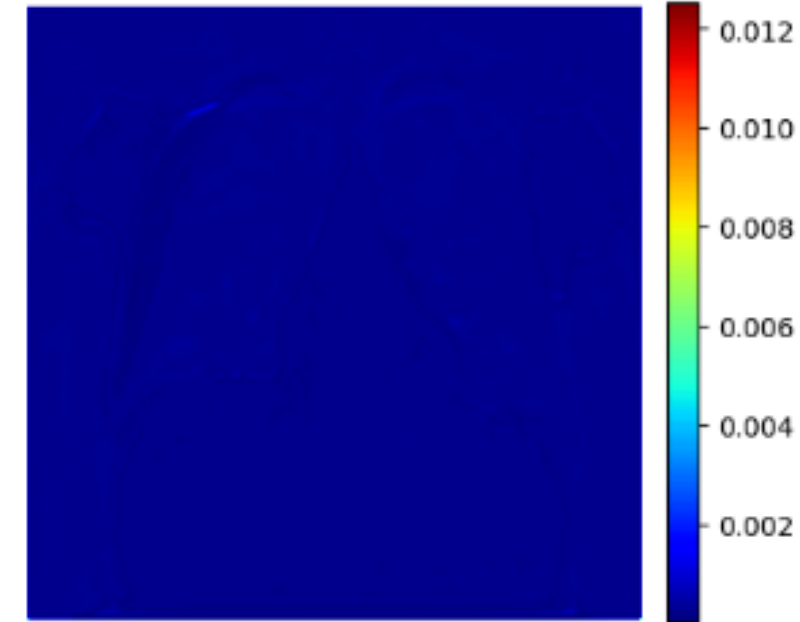
1.2.276.0.7230010.3.1.4.8323329.4403.1517875182.670658



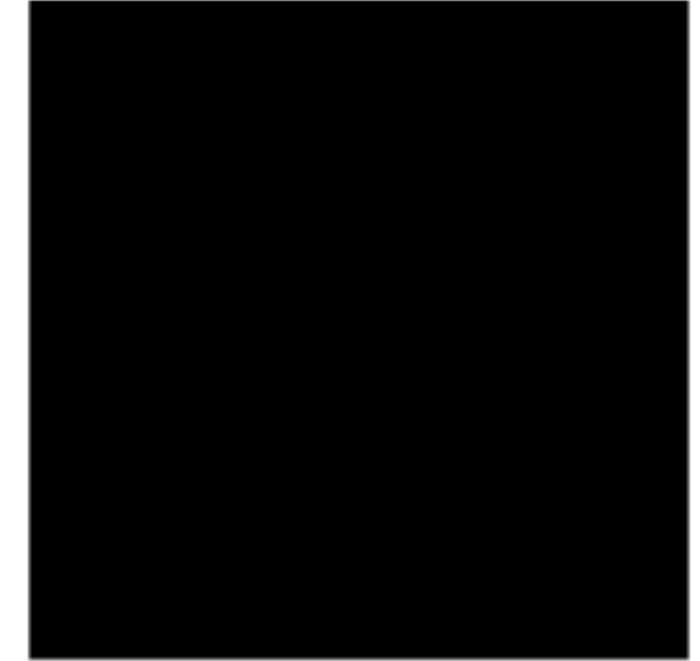
Ground Truth (mask)



Prediction (prob)



Prediction (thr=0.3)



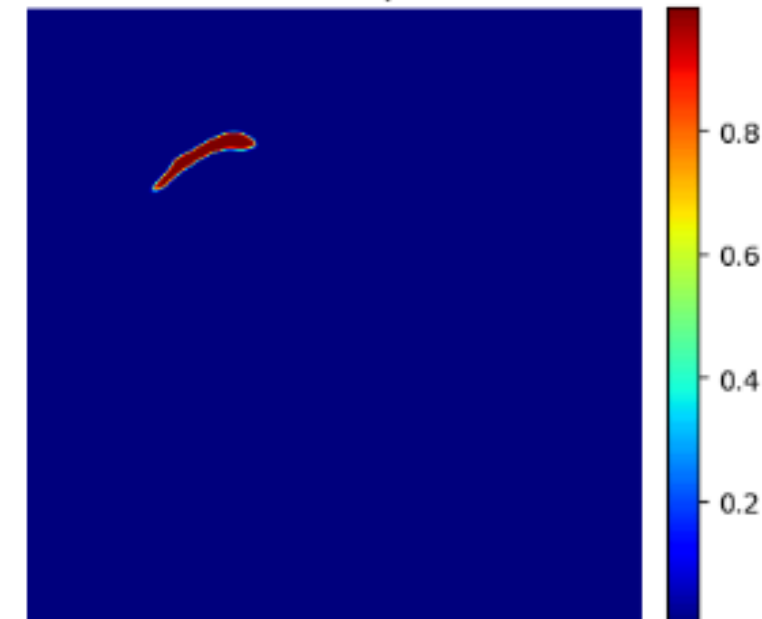
Input
1.2.276.0.7230010.3.1.4.8323329.3746.1517875179.258086



Ground Truth (mask)



Prediction (prob)

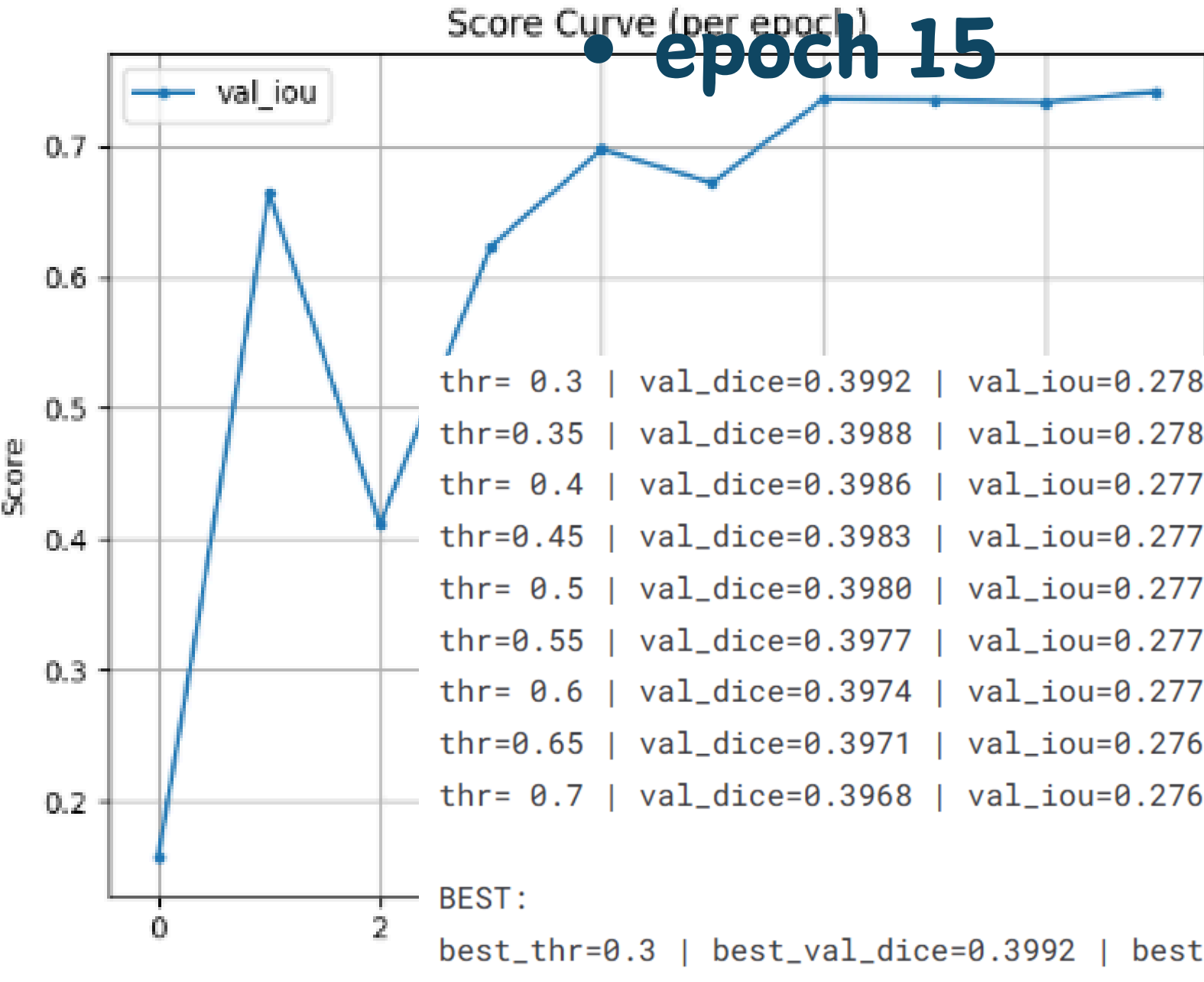
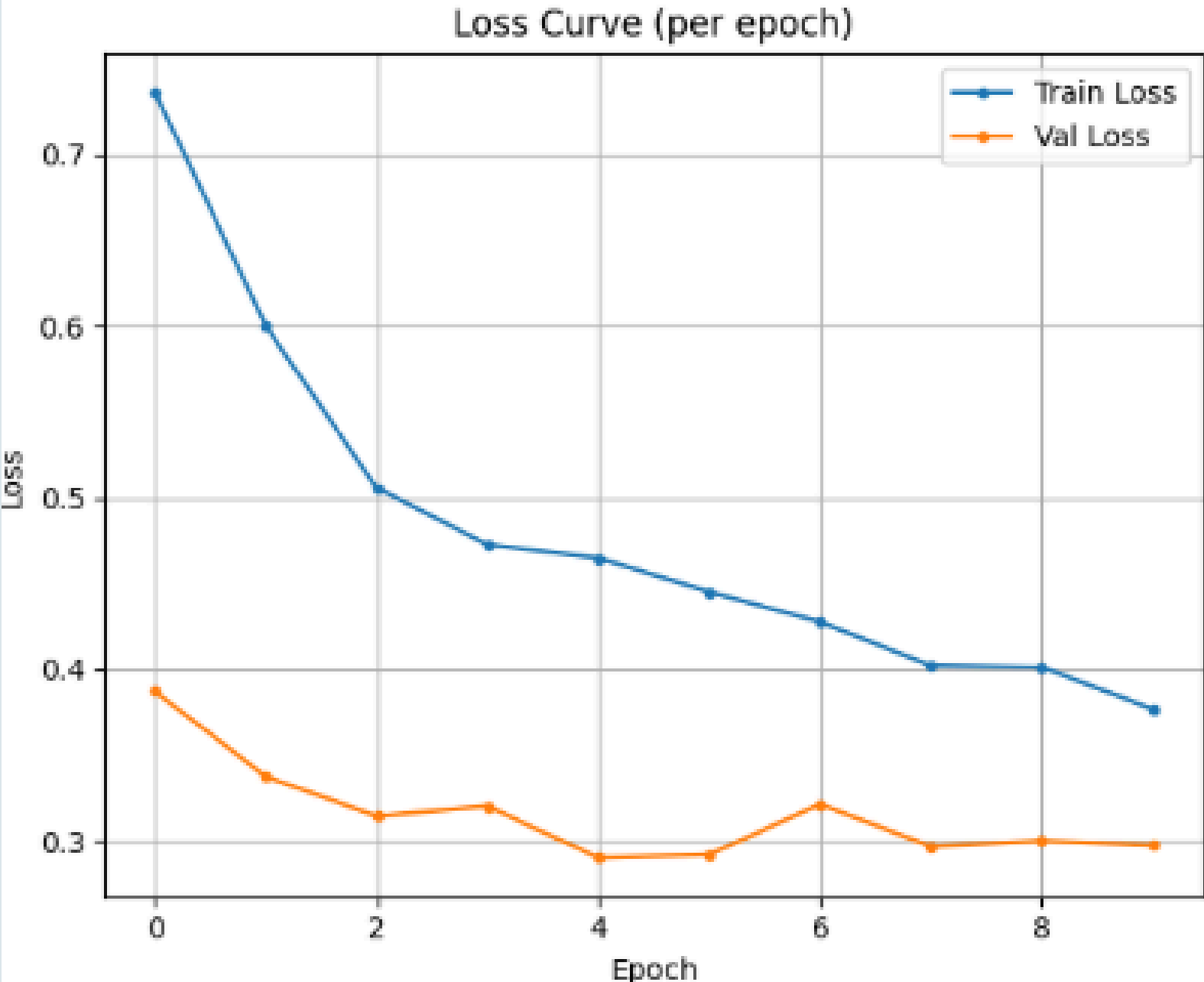


Prediction (thr=0.3)



最大問題:修正後

• resnet50



• epoch 15



notebook5131c5ce19 - Version 3

Complete (after deadline) · 1d ago

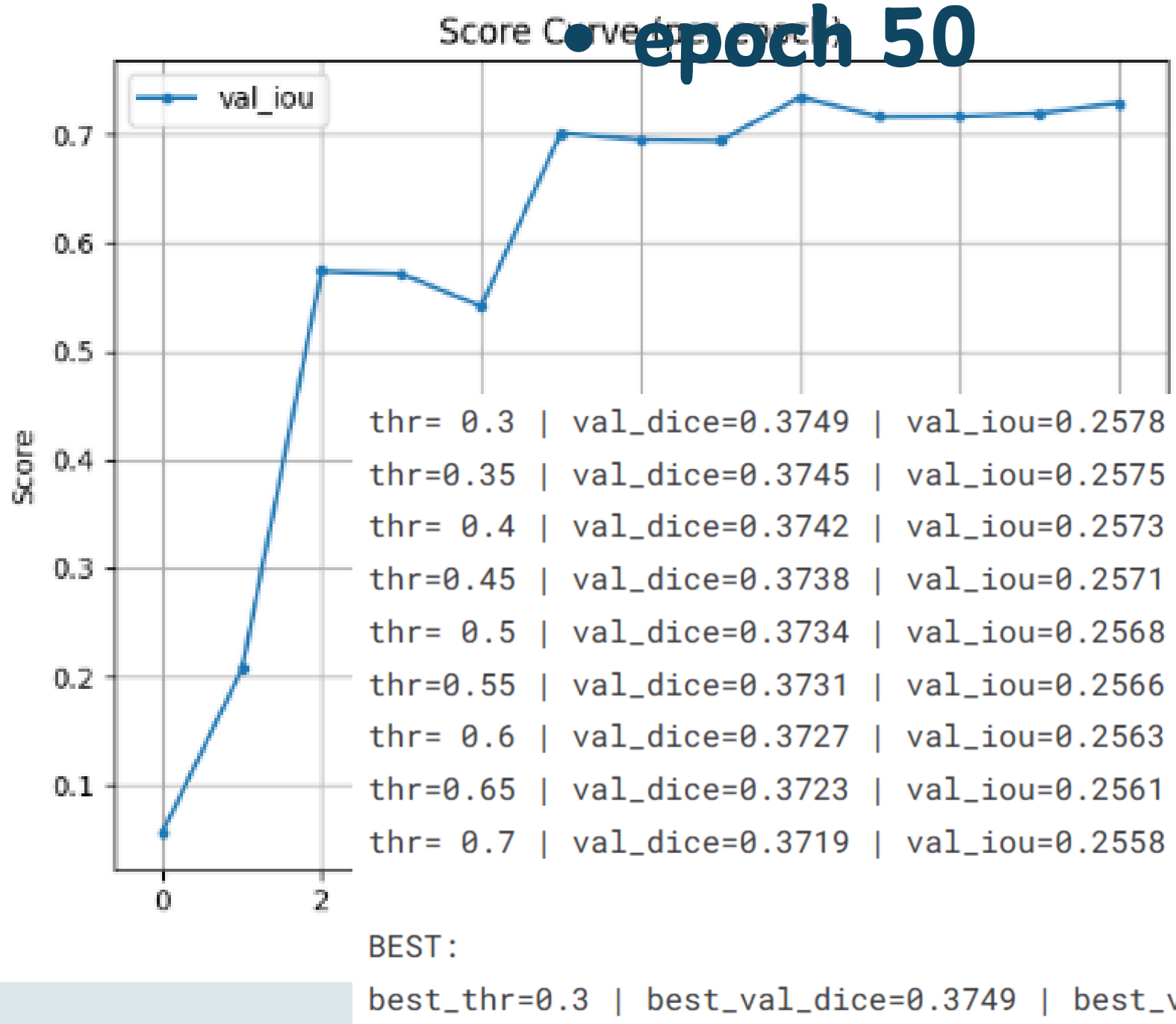
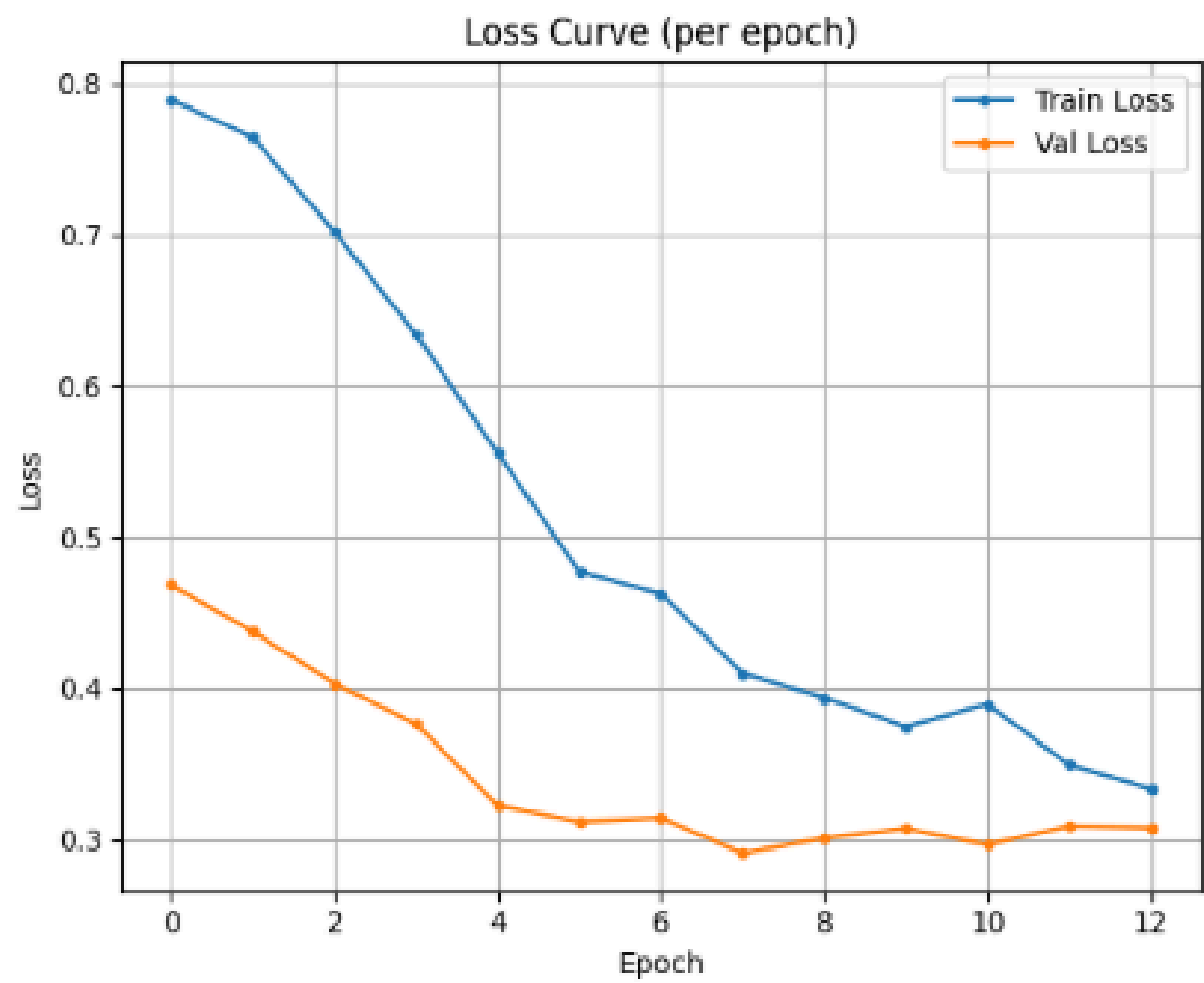
0.7740

0.6000

最大問題:修正後

• resnet50

• epoch 50



notebook5131c5ce19 - Version 5

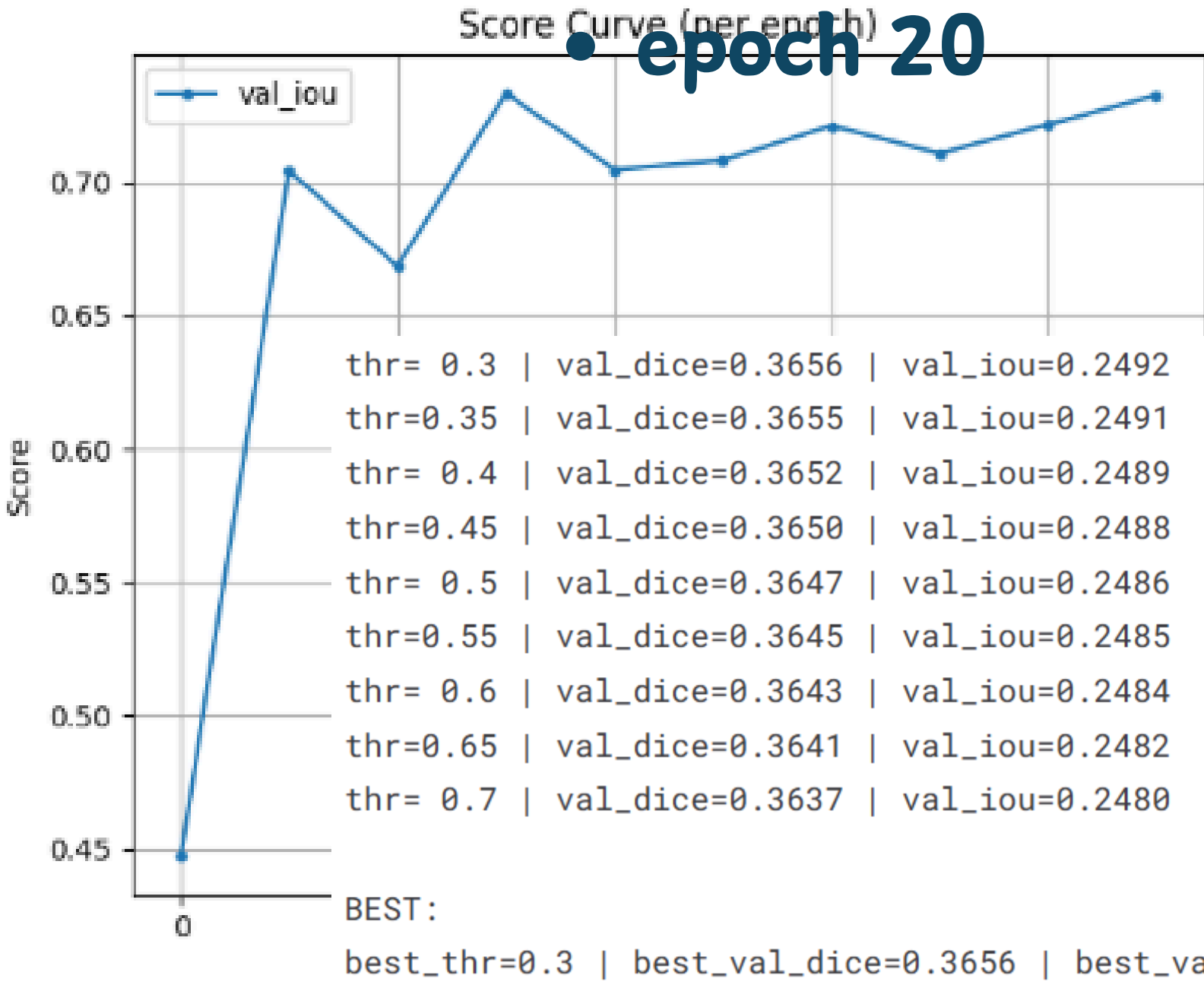
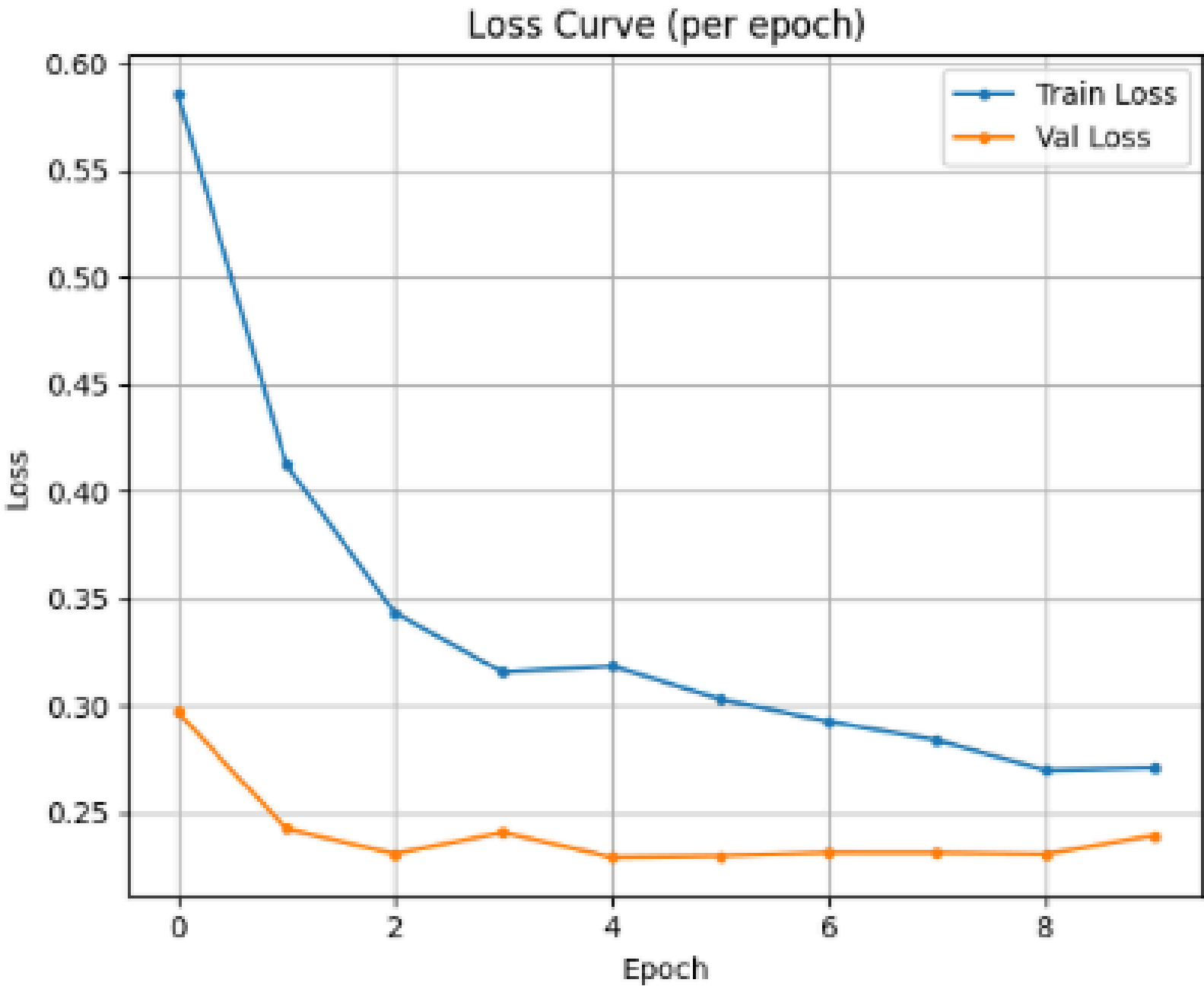
Complete (after deadline) · 19h ago

0.7740

0.6000

最大問題:修正後

• efficientnet-b4



• epoch 20

0.7372

0.6000



notebook5131c5ce19 - Version 4

Complete (after deadline) · 1d ago

其次問題

- thr = ?
 - 原本直接設定0.3，但我們有best_thr可用，於是發現後修正為thr = best_thr
- 我們在訓練時 (Train) 只用了 medical_intensity_pipeline (0~1 歸一化)；但在測試時 (Test, Cell 19) 卻額外加了 preprocess_fn (標準化 mean/std)，我們應該要刪除或者統一。

```
# 在 PnuemoTestDataset.__getitem__ 中：  
# if preprocess_fn is not None:      <-- 刪除這行  
#     img = preprocess_fn(img)      <-- 刪除這行
```

總結

所以結論是？

目標與達成

- 具體交付項目：
 - 提交Kaggle，預期排名前50->實作未達成(約排於300)
 - 產出一個高效能的氣胸分割模型(模型權重檔)->達成!(有產出)
 - 繳交一份完整的實作報告：程式碼公開連結

309



submission_efficientnet-b4.csv

Complete (after deadline) · 3d ago

0.7740

0.6000

0.7740

310

19

Michael.wang



0.7740

目標與達成

- 其他延伸

- 由於我們怎麼做，最高上限皆為0.7740，因此我們嘗試參考其他人的於 notebook 介紹的辦法來實作，而獲得了以下成績，所以我們認為我們一定有哪裡可以改進。

189	▼ 28	Heng CherKeng		0.8256
 submission.csv Complete (after deadline) · 8h ago				
190	▼ 153	Mugur Tulan		0.8256

- 由於模型跑一次(30epoch)需約7~9小時，且可能會有OOM的問題，因此如若有更多資源，此次實作結果將更良好。

未來展望

還有好多問題沒有解決

未解決問題與可能辦法

- 優化後處理策略 (Refining Post-processing Thresholds)
 - 目前的模型設定將像素總和 (Sum of Pixels) 小於 500 的預測結果視為雜訊並予以捨棄。然而，這可能導致微小的氣胸病灶被錯誤過濾 (False Negative)。未來可嘗試調降此閾值 (例如降至 200)，或結合連通域分析 (Connected Component Analysis) 來更精細地保留小型病灶。

未解決問題與可能辦法

- 模型集成 (Model Ensembling)
 - 單一隨機種子 (Seed=42) 可能導致訓練結果受初始化參數影響。未來可採用模型集成策略，使用不同的隨機種子訓練多個模型，並對預測結果進行加權平均 (Weighted Averaging)。這能有效降低單一模型的變異數，提升預測的穩定性與準確度。

未解決問題與可能辦法

- 進階資料增強 (Advanced Data Augmentation)
 - 受限於計算資源與訓練時間，本階段僅使用基礎的縮放與標準化。未來或許可以導入 Albumentations 函式庫進行幾何增強（如隨機旋轉）與光度增強（如隨機亮度對比調整）。這能模擬更多樣的 X 光拍攝條件，強健模型對抗過曝或擺位不正影像的泛化能力。

兩階段偵測框架

- 鑑於資料集中負樣本（無氣胸）比例極高，單一分割模型極易在健康影像上產生雜訊，導致產生大量偽陽性 (False Positive)，這也是目前模型難以突破「全預測 -1」基準分的主因。
- 未來擬改採兩階段策略：
 - 第一階段 (Classification)：訓練一個高準確率的分類器（如 EfficientNet），專門篩選出陽性樣本。
 - 第二階段 (Segmentation)：僅針對被分類為「陽性」的影像進行病灶分割。
- 此策略將能有效過濾無病灶樣本，避免模型在正常組織上產生誤判，同時讓分割模型專注學習病灶的邊界特徵，預期能顯著提升整體的 Dice Score。

小組分工

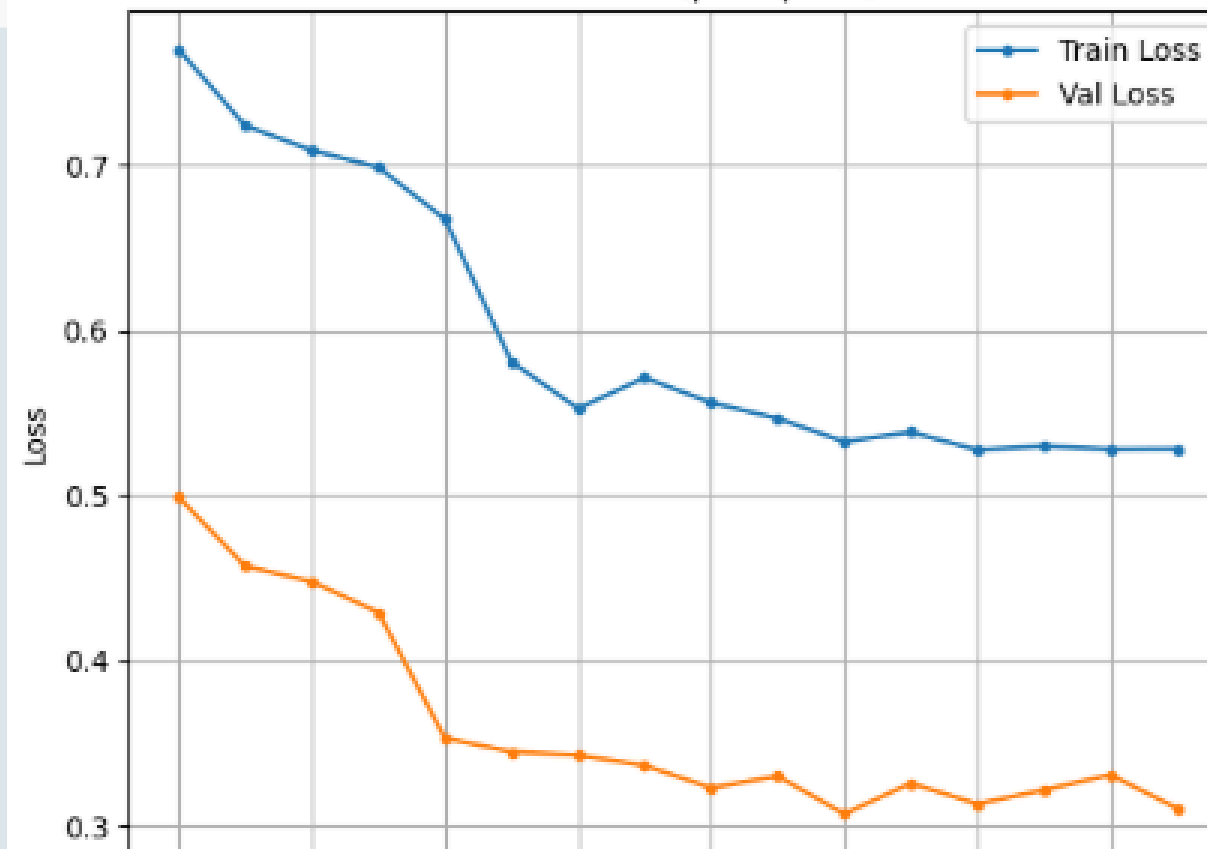
團隊成員	工作內容
劉姮廷	debug、訓練、問題排查及嘗試解決、其他部分PPT製作及整體排版
胡樂麒	資料探索、前處理、訓練、前處理部分PPT製作
邱庭俞	模型架構、訓練、debug、模型架構PPT製作
李怡萱	初版pipeline、backbone選擇、訓練
蔡勇濱	終版pipeline、backbone選擇、kaggle繳交
所有人皆負責報告	

其他結果

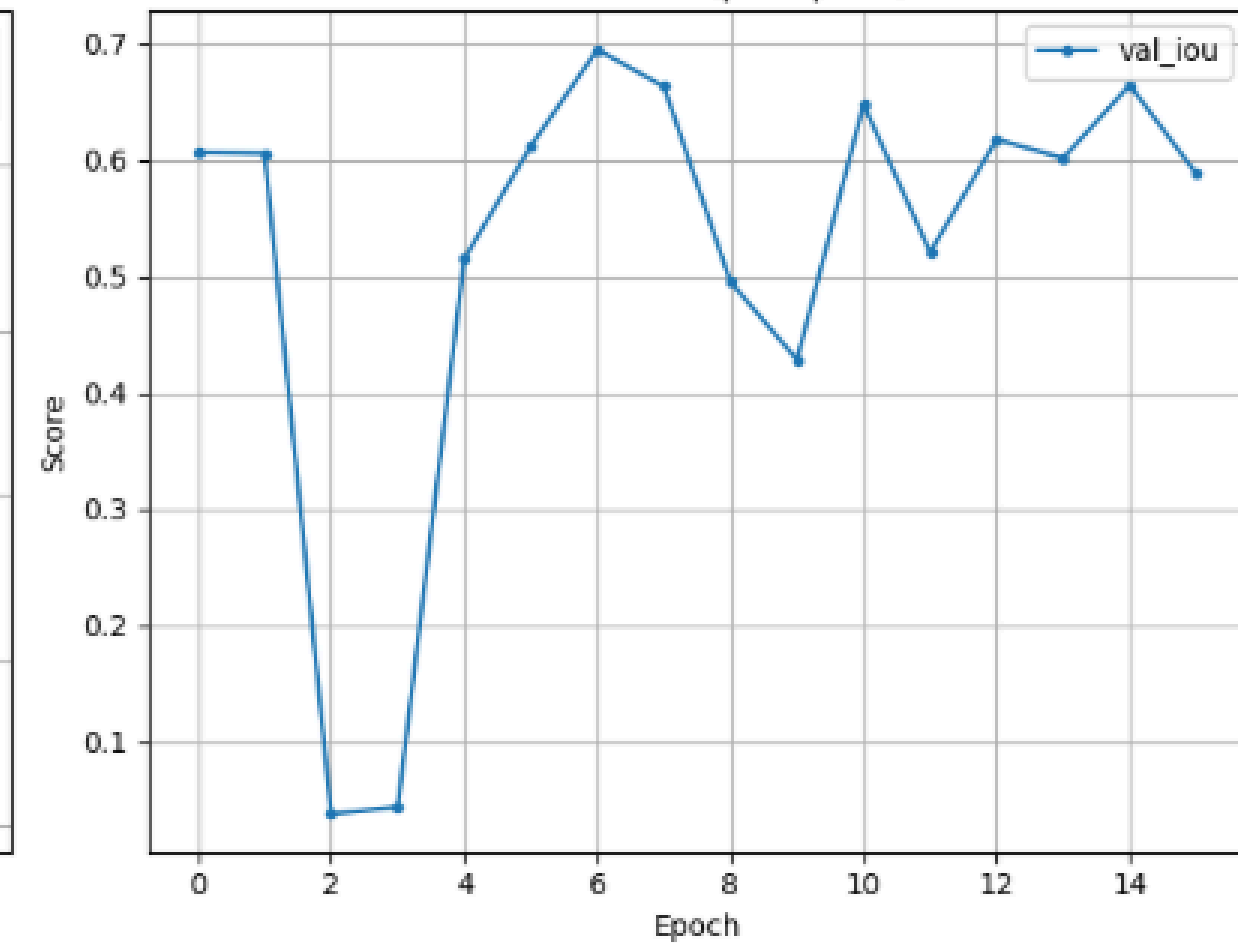
我們還有做其他backbone

se_resnext50_32x4d

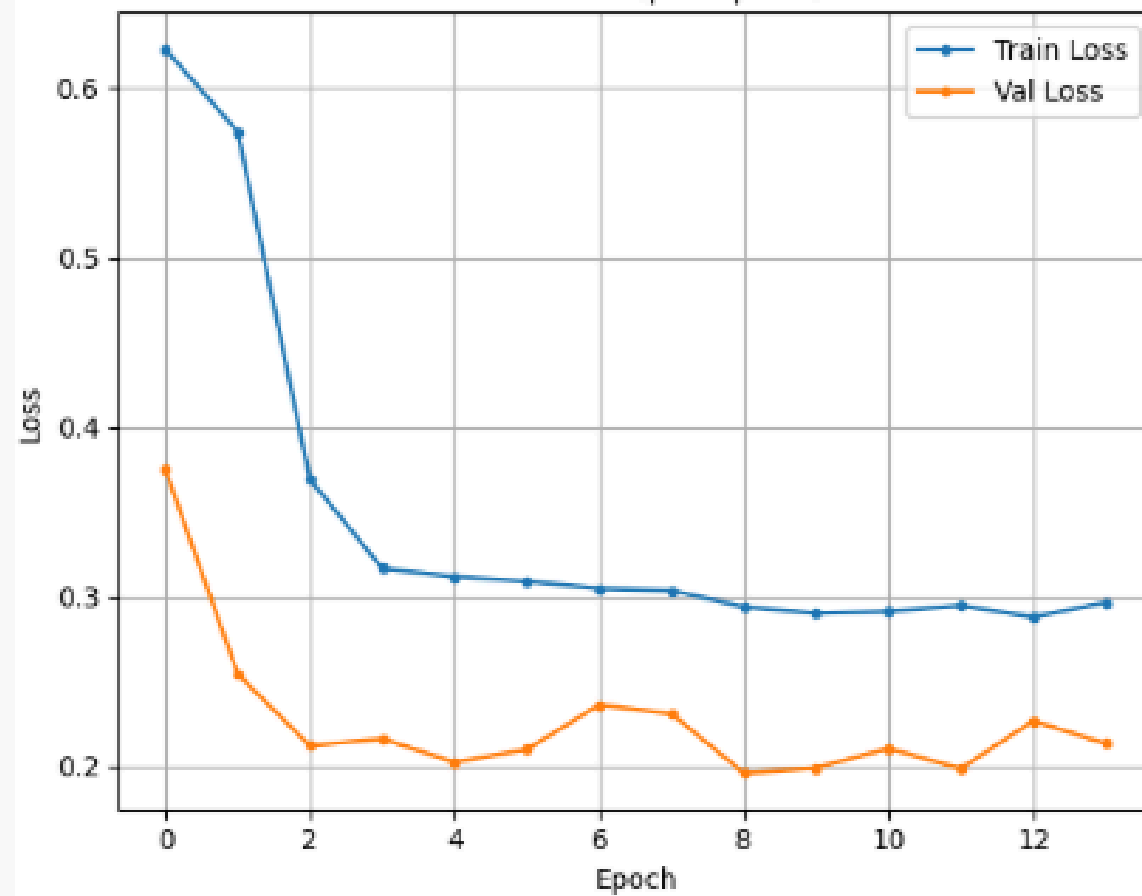
Loss Curve (per epoch)



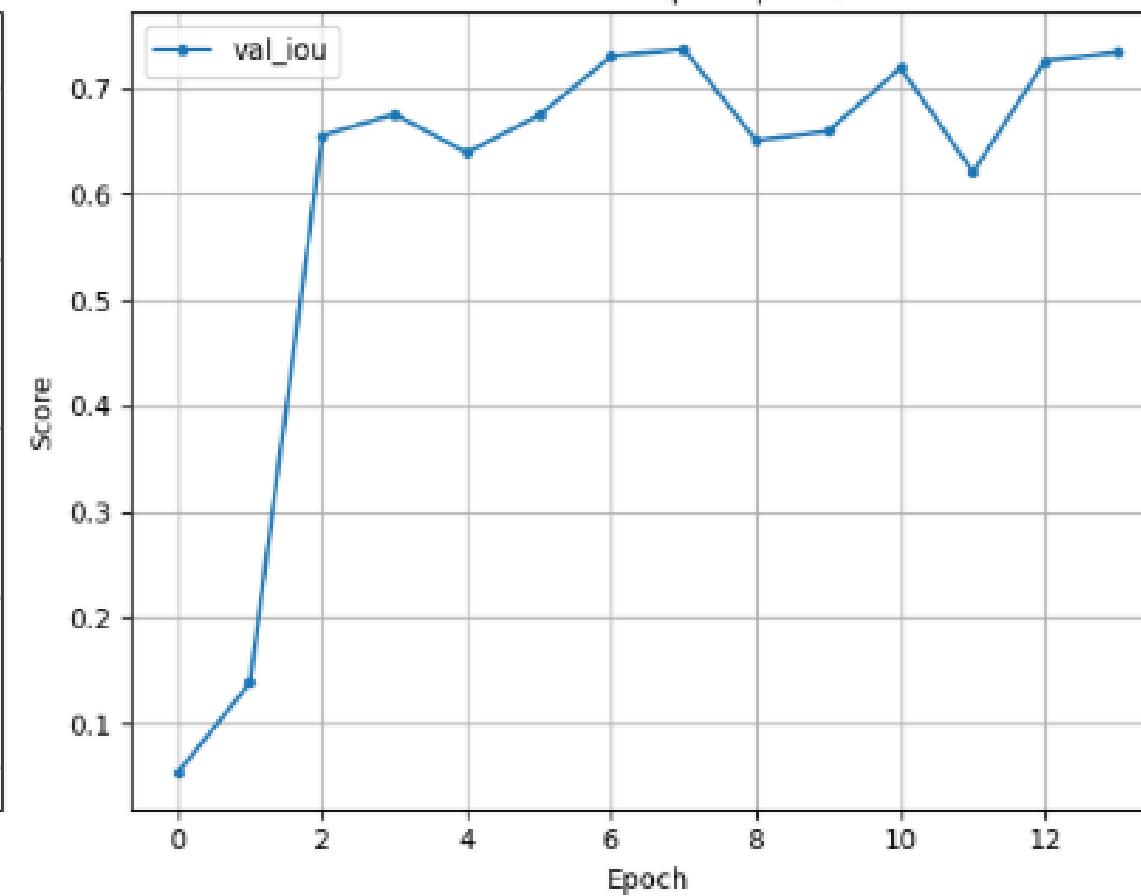
Score Curve (per epoch)



Loss Curve (per epoch)

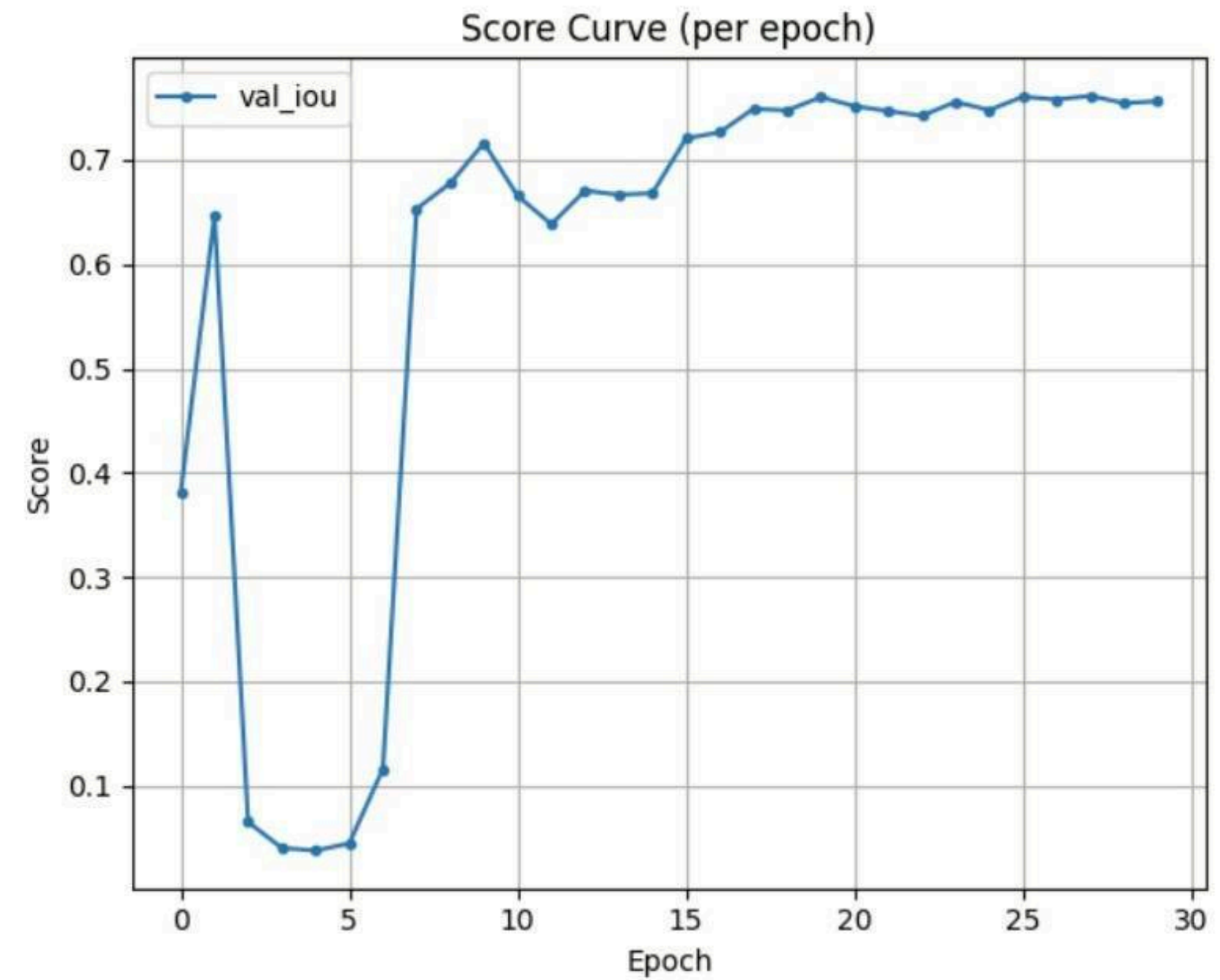
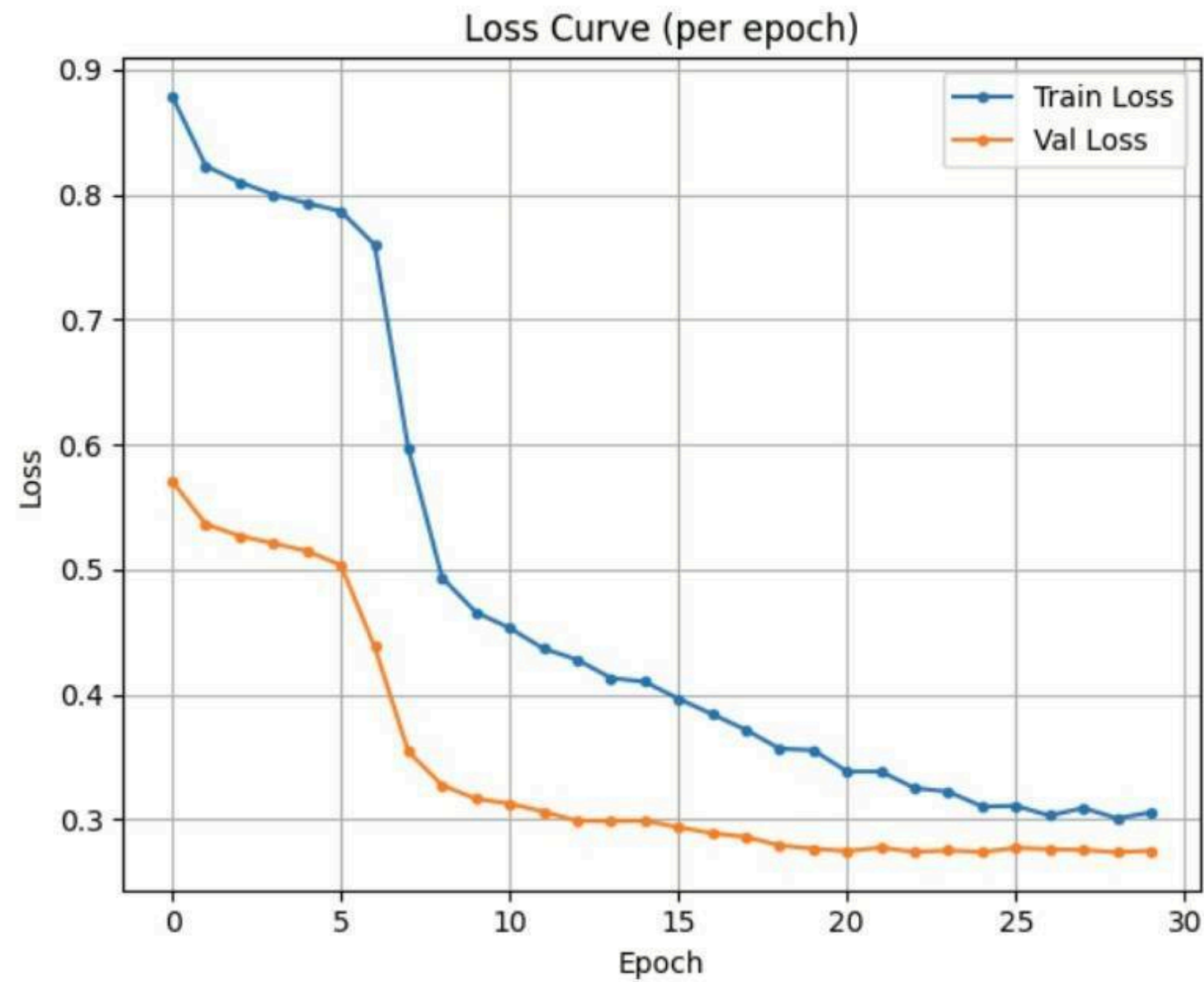


Score Curve (per epoch)

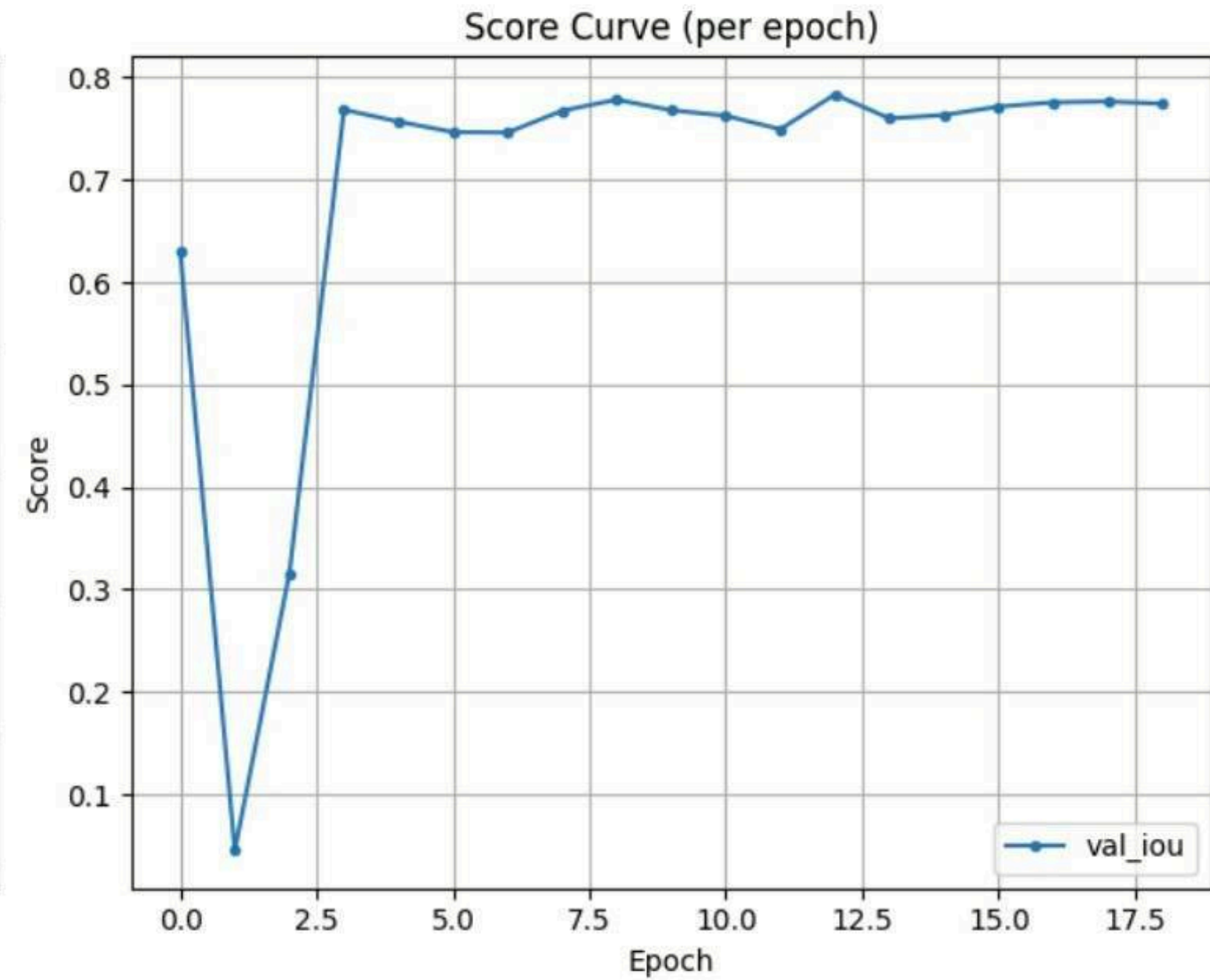
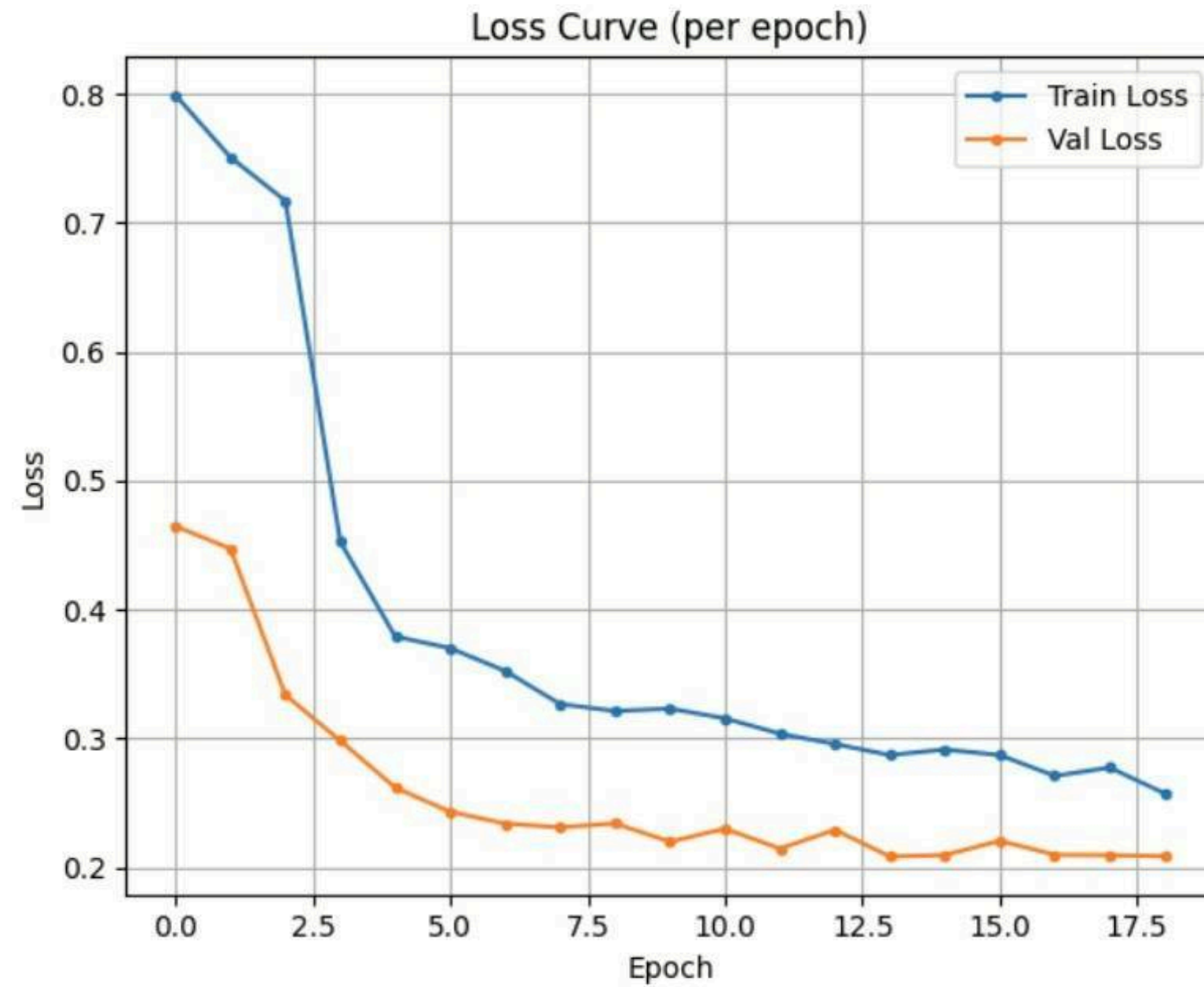


- thr改為0.5
- 加入旋轉

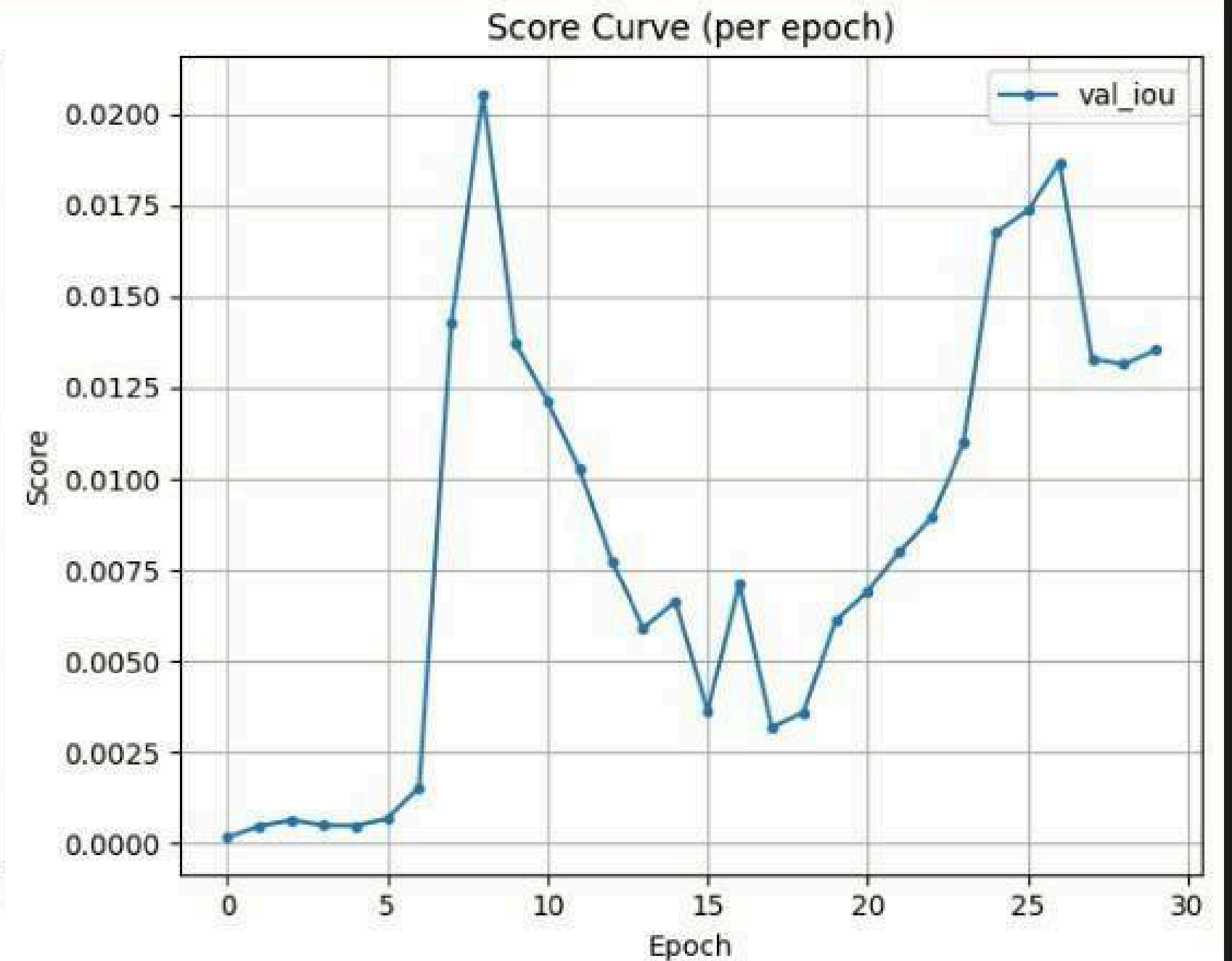
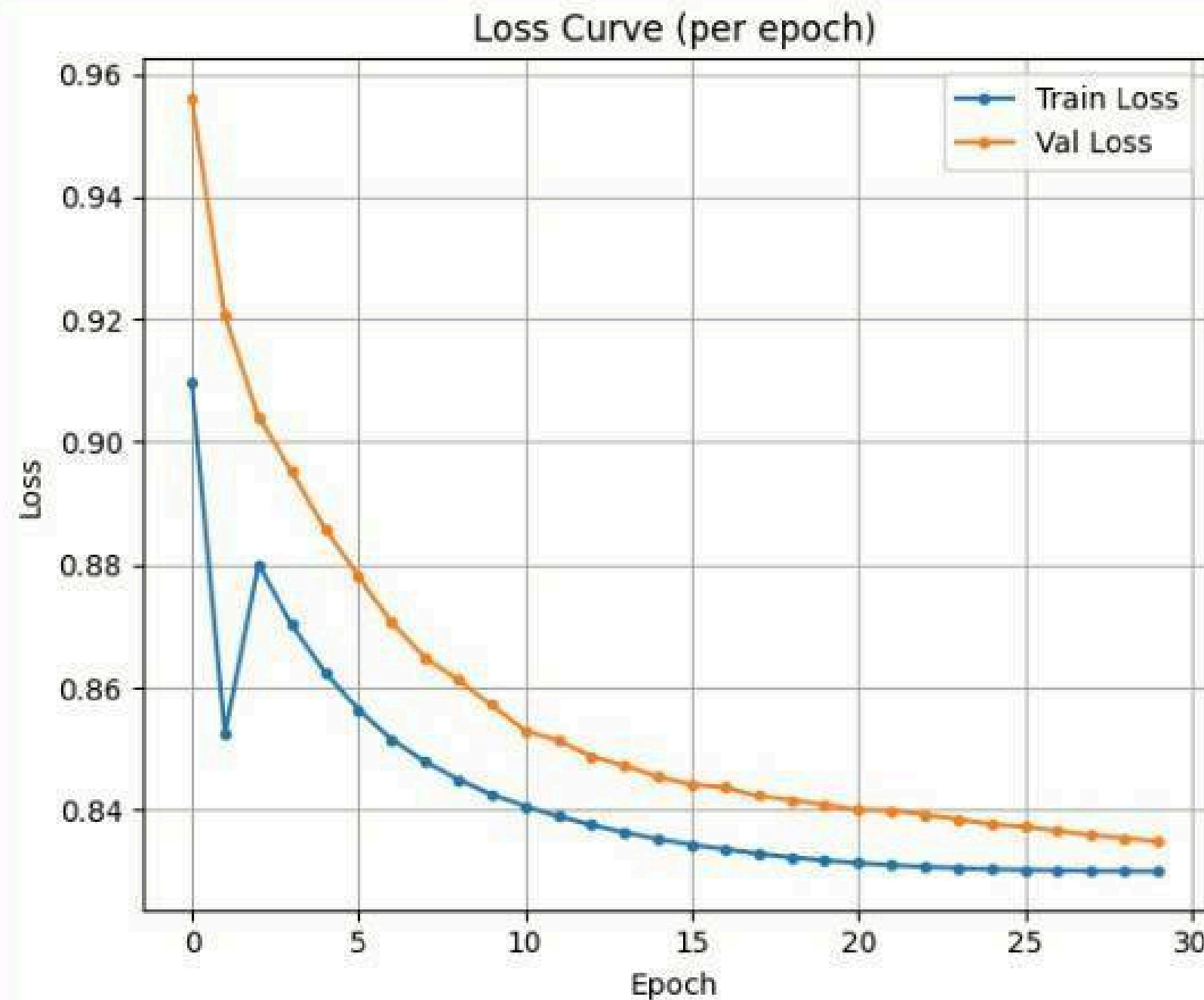
efficientnet-b0



efficientnet-b1



efficientnet-b4



submission_efficientnet-b4.csv

Complete (after deadline) · 3d ago

0.7740

0.6000

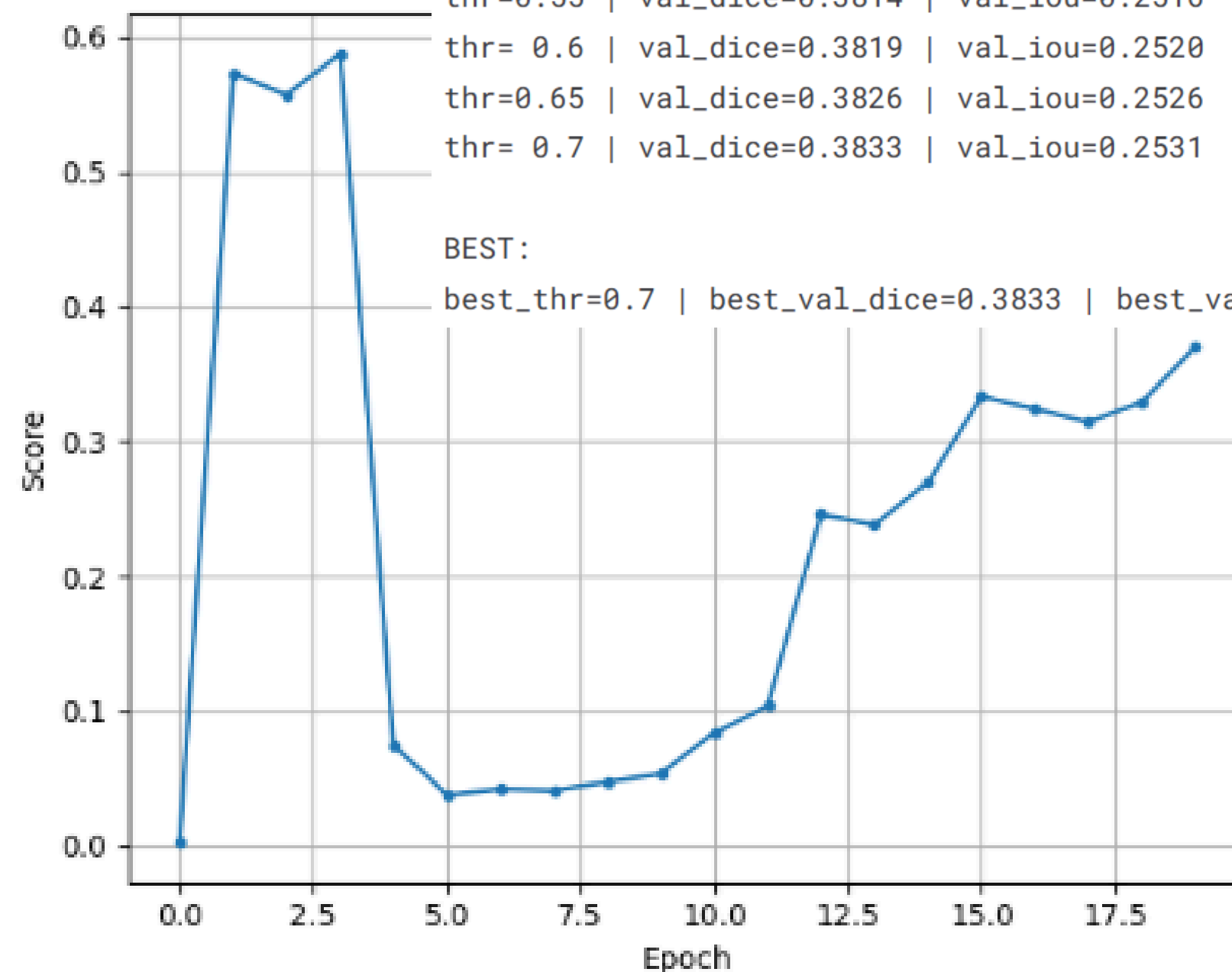
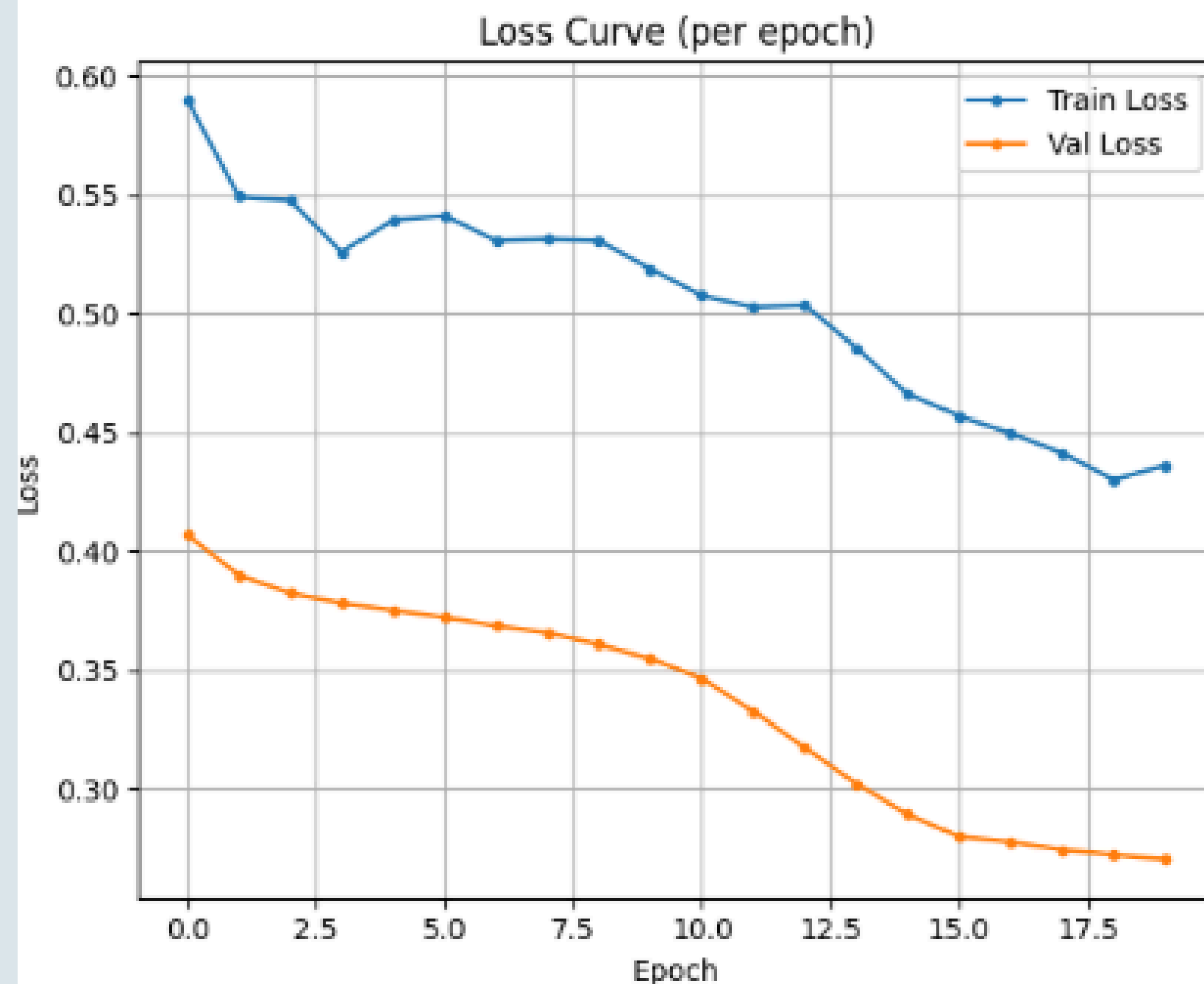
• 30epoch

efficientnet-b5

thr= 0.3 | val_dice=0.3784 | val_iou=0.2492
thr=0.35 | val_dice=0.3792 | val_iou=0.2498
thr= 0.4 | val_dice=0.3797 | val_iou=0.2502
thr=0.45 | val_dice=0.3800 | val_iou=0.2505
thr= 0.5 | val_dice=0.3809 | val_iou=0.2512
thr=0.55 | val_dice=0.3814 | val_iou=0.2516
thr= 0.6 | val_dice=0.3819 | val_iou=0.2520
thr=0.65 | val_dice=0.3826 | val_iou=0.2526
thr= 0.7 | val_dice=0.3833 | val_iou=0.2531

BEST:

best_thr=0.7 | best_val_dice=0.3833 | best_val_iou=0.2531



- thr改為0.5
- 加入旋轉

... Epoch 18/19

1243/2591 0:07:06 • 0:07:42 2.92it/s v_num: 0.000 val_loss: 0.001
val_iou: 1.000 train_loss: 0.001



ting222 - Version 6

Complete (after deadline) · 2d ago

0.7490

.....



Thank you



.....